ServiceRegistry

Interface Design Description

Eclipse Arrowhead 5.2.2 | core system | Revision 2026-11-09 | REVIEW DRAFT

Purpose and document structure

This document consolidates the system interface designs. Detailed Java service/transport IDDs are retained as separate appendices, followed by available Go and Rust interface descriptions. The current cross-language sections below precede the detailed implementation appendices. Appendix dates and release labels belong to the retained baseline, not to a newly qualified release. Existing descriptions have been assembled from the primary trees; no live interoperability or semantic-parity claim is made.

Java implementation

Primary folder: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceregistry. 11 detailed Java IDD source document(s) are included in the appendices. Their architecture, operation parameters, models, transport bindings, access rules and errors remain associated with Java. They must not be read as automatically implemented by the ports.

Go implementation

Primary folder: Eclipse-Arrowhead-go/ah5-core-go/serviceregistry.

ServiceRegistry - Go 5.2.2

Standalone core system, reorganized on 2026-10-28 following the Eclipse-Arrowhead-java distribution layout. Build and operate from this folder; no original Arrowhead checkout or sibling source module is required. The executable is bin/ServiceRegistry. Entry point: .. Module: arrowhead/serviceregistry. See DEPLOYMENT.md for configuration and standalone container use. Existing compatibility limits remain in the copied upstream documentation and UPSTREAM-README.md. Existing role selection and API behavior are retained. Documentation and license notices are included in this folder. Source provenance and import-only rewrites are listed in the distribution's SOURCE-MANIFEST.json. No running cloud or upstream source was changed.

Listening port

Default 8443 (server.port in application.properties). Override with server.port, SERVER_PORT=N or -server.port N; the host is server.address. See [LISTENING-PORTS.md](../../LISTENING-PORTS.md) for precedence and all systems.

Native Go registry design and protocol contract

Arrowhead 5.2.2 ServiceRegistry (Go)

A Go implementation of the Arrowhead 5.2.2 ServiceRegistry core system, modelled on the Java Spring reference implementation (eclipse-arrowhead/ah5-core-java-spring). HTTP/TLS handling uses the standard library; the optional native monitor uses github.com/rabbitmq/amqp091-go. State is held in memory with an atomic JSON snapshot file. See the parent repository's compatibility report for limitations of the complete Go core-system bundle.

Build, test, run

go build -o serviceregistry . go test ./... ./serviceregistry -config application.properties Docker: docker build -t ah5-serviceregistry-go . Configuration uses the reference property names (see application.properties); each property can be overridden by an environment variable (SERVER_PORT, DOMAIN_NAME, AUTHENTICATION_POLICY, ...).

API (base path `/serviceregistry`)

Area	Endpoints
Monitor	GET /monitor/echo
Device discovery	POST /device-discovery/register, POST /device-discovery/lookup, DELETE /device-discovery/revoke/{name}
System discovery	POST /system-discovery/register, POST /system-discovery/lookup[?verbose], DELETE /system-discovery/revoke
Service discovery	POST /service-discovery/register, POST /service-discovery/lookup[?verbose], DELETE /service-discovery/revoke/{instanceId}
Management	/mgmt/{devices,systems,service-definitions,service-instances,interface-templates} (POST create, PUT update where applicable, DELETE with names= / serviceInstances=) and POST .../query
General management	POST /general/mgmt/logs, GET /general/mgmt/get-config?keys=

Implemented behaviour: the three authentication policies (declared, certificate, outsourced), the management filter (sysop-only / whitelist / authorization) and the blacklist filter, restricted service discovery (unrestrictedDiscovery metadata), interface policies (open / extendable / restricted) with default templates, property validators, the metadata-requirement DSL, cascading deletes, pagination, Arrowhead error bodies, and self-registration of the ServiceRegistry and its five services at startup (revoked on shutdown; disabled by standalone.mode=true).

Outsourced authentication with mutual TLS

With authentication.policy=outsourced, server.ssl.enabled=true and server.ssl.client-auth=need every request needs a local-cloud client certificate and an identity token (or the Authentication system's authenticator key):

- The token's system must be the system named by the client certificate CN (token-to-certificate binding), so a stolen

token cannot be used from another certificate.

- A lookup of exactly the identity service is allowed without a token, as in the Java framework: a system must find

Authentication before its first login. The client certificate is still required.

- The registry logs in itself with authenticator.credentials (system name ServiceRegistry), reuses its token for the

identity-verify calls to Authentication and logs in again when the token is refused. Authentication's authenticator key is checked against authenticator.secret.keys.

- Its own services are registered as generic_https with policy CERT_AUTH when client certificates are required.

sysMonitor (native publisher)

With sysmonitor.enabled=true the registry registers its own sysMonitor 1.0.0 offer (generic_amqps, CERT_AUTH, metadata publisherMode=native) and publishes echo, power and status observations every

sysmonitor.publish.interval seconds (default 30) to the AMQP topic exchange sysmonitor.exchange with routing key <sysmonitor.local.cloud>.sysmonitor.ServiceRegistry.ServiceRegistry.<type> (contract SysMonitor-AMQP-IDD-5.2.2). Messages are persistent and confirmed by the broker. The client (sysmonitor.go, github.com/rabbitmq/amqp091-go) connects with mutual TLS and SASL EXTERNAL, so the broker user is the certificate CN; echo reports degraded when the snapshot file is not writable. The registry also seeds the interface templates generic_amqp and generic_amqps (properties accessAddresses, accessPort, exchange, baseRoutingKey, operations).

Property	Default	Meaning
sysmonitor.enabled	false	Register and publish sysMonitor.
sysmonitor.broker.address / .port	amqp-broker / 5671	AMQP broker (TLS).
sysmonitor.exchange	arrowhead	AMQP topic exchange.
sysmonitor.local.cloud	TestCloud	Local-cloud routing-key word.
sysmonitor.publish.interval	30	Seconds between publication cycles.
sysmonitor.ssl.cert-file, .key-file, .ca-file	-	PEM client certificate, key and CA.

Environment overrides follow the usual mapping (SYSMONITOR_ENABLED, SYSMONITOR_BROKER_ADDRESS, ...).

Additional ingresses of the registry's own services

service.registry.extra.interfaces (environment SERVICE_REGISTRY_EXTRA_INTERFACES, default []) is a JSON list of extra interfaces that the registry adds to its own service instances every time it registers itself, for example a private mTLS gateway: Base path and operations are copied from the service's native interface; protocol defaults from the template name and policy to NONE. Because the entries are part of the configuration, they survive registry restarts (the registry revokes its own instances on shutdown). The Java ServiceRegistry of ah5-core-java-spring supports the same property.

Differences from the Java reference

- No MQTT API.

- JSON-file persistence instead of MySQL.

- Empty metadata is omitted from responses; whole-number floats lose .0 after a persistence round trip.

- Lookup filters evaluate each interface independently (the Java code leaks policy/template/property

flags between interfaces, treated as a bug); the address-type filter is applied independently of property requirements; mandatory template properties are checked uniformly.

- addressType in management queries is case-insensitive.

- String-set properties keep input order with duplicates removed.

- Outbound calls to ConsumerAuthorization and Blacklist services are best-effort implementations of

the documented verify/check operations and have not been tested against the real services. Identity login and verification are verified in the Arrowhead test cloud (Arrowhead-test-cloud-go).

- Tested with unit/httptest integration tests and in the Go test cloud.

- The parent Go core bundle additionally tests a temporary native registry process with the new

Go orchestration and authorization implementations. Management authorization now sends the v5.2.2 targetType / target / scope fields and registered operation names; outsourced tokens grant sysop access only when Authentication returns sysop=true, not merely for a Sysop name.

Detailed implementation evidence

0 native Go IDD document(s) are included in the appendices. Their protocol and storage descriptions apply to that implementation only. Current primary-folder ownership and current listening-port notes above supersede historical paths or example ports in those retained documents.

Rust implementation

Primary folder: Eclipse-Arrowhead-rust/ah5-core-rust/serviceregistry.

ServiceRegistry (Rust)

Arrowhead 5.2.2 core system. src/main.rs is built as binary serviceregistry by the arrowhead-core-rust crate in ... serviceregistry.json and documentation/ link to ../examples and ../documentation/ServiceRegistry.

Detailed implementation evidence

1 native Rust IDD document(s) are included in the appendices. Their protocol and storage descriptions apply to that implementation only. Current primary-folder ownership and current listening-port notes above supersede historical paths or example ports in those retained documents.

Build and state ownership

Core entry points are built by arrowhead-core-rust; the ordinary support entry points are built by arrowhead-support-rust using the shared support runtime. LocalCloudManager has its own crate. Per-system documentation links may point into the shared documentation catalog. Independent identity and state configuration is still required. Read the native appendix for persistence and restart semantics.

Cross-language qualification

Identical system names or HTTP paths do not establish interoperability. For the selected implementation, verify registered service versions and metadata, method/path and payload models, null and optional-field handling, success/error bodies, authentication and per-operation authorization, timeout/retry behavior and persistence across restart. Java MQTT/AMQP descriptions apply to a port only where its own implementation evidence explicitly supports them. Missing transports must not be advertised. State conversion from Java SQL to native snapshots, secret provisioning, backup/restore and rollback require implementation-specific procedures. No data migration, deployment or cloud reconfiguration was performed for this document set.

Detailed appendices and provenance

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceregistry/documentation/v5.2.2/ServiceRegistry-v5.2.2-deviceDiscovery-HTTP-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceregistry/documentation/v5.2.2/ServiceRegistry-v5.2.2-deviceDiscovery-MQTT-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceregistry/documentation/v5.2.2/ServiceRegistry-v5.2.2-generalManagement-HTTP-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceregistry/documentation/v5.2.2/ServiceRegistry-v5.2.2-generalManagement-MQTT-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceregistry/documentation/v5.2.2/ServiceRegistry-v5.2.2-serviceDiscovery-HTTP-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceregistry/documentation/v5.2.2/ServiceRegistry-v5.2.2-serviceDiscovery-MQTT-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceregistry/documentation/v5.2.2/ServiceRegistry-v5.2.2-serviceRegistryManagement-HTTP-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceregistry/documentation/v5.2.2/ServiceRegistry-v5.2.2-serviceRegistryManagement-MQTT-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceregistry/documentation/v5.2.2/ServiceRegistry-v5.2.2-sysMonitor-AMQP-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceregistry/documentation/v5.2.2/ServiceRegistry-v5.2.2-systemDiscovery-HTTP-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceregistry/documentation/v5.2.2/ServiceRegistry-v5.2.2-systemDiscovery-MQTT-IDD.tex

- Rust: Eclipse-Arrowhead-rust/ah5-core-rust/documentation/ServiceRegistry/ServiceRegistry-HTTPS-IDD.tex

The package sources/ folder retains the selected LaTeX, figures, bibliography, language README/build evidence and source snapshots for newly described contracts. source-manifest.json records SHA-256 hashes of original inputs. build-report.json records whether an appendix used an existing newer PDF or was compiled from source.

deviceDiscovery HTTP/TLS/JSON

Interface Design Description

Service ID: *"deviceDiscovery"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `deviceDiscovery` service of the ServiceRegistry system.

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation register . . . 4
2.2 operation lookup . . . 5
2.3 operation revoke . . . 5

3 Data Models **7**
3.1 struct Address . . . 7
3.2 struct DeviceListResponse . . . 7
3.3 struct DeviceLookupRequest . . . 7
3.4 struct DeviceRequest . . . 7
3.5 struct DeviceResponse . . . 7
3.6 struct ErrorMessage . . . 8
3.7 struct MetadataRequirement . . . 8
3.8 Primitives . . . 8

4 References **9**

5 Revision History **10**
5.1 Amendments . . . 10
5.2 Quality Assurance . . . 10

1 Overview

This document describes the `deviceDiscovery` service interface of the ServiceRegistry system, which provides registration and lookup of the devices of the local cloud: the physical or virtual hosts, with their network addresses and metadata, on which the systems run.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `deviceDiscovery` service interface

This document provides the Interface Design Description IDD to the *deviceDiscovery – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `deviceDiscovery` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https` (`generic_http` when TLS is disabled), access port 8443 and base path `/serviceregistry/device-discovery`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`register`	POST	`/register`
`lookup`	POST	`/lookup`
`revoke`	DELETE	`/revoke/{name}`

2 Service Operations

The interfaces of the `deviceDiscovery` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `deviceDiscovery` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json`. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

2.1 POST /serviceregistry/device-discovery/register

Operation: register
Input: DeviceRequest
Output: DeviceResponse

Registers a device with its addresses and metadata. If a device with the same name and the same data exists, it is returned unchanged (200); a new device is created (201). A device with the same name and different data is refused.

A successful call returns `201 Created` for a new entry, `200 OK` if an identical entry already existed with the output in the body.

```
POST /serviceregistry/device-discovery/register HTTP/1.1
Host: serviceregistry:8443
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "name": "Raspberry1",
  "metadata": {
    "location": "hall-1"
  },
  "addresses": [
    "string"
  ]
}
```

Listing 1: An register invocation.

```
{
  "name": "Raspberry1",
  "metadata": {
    "location": "hall-1"
  },
  "addresses": [
    {
      "type": "IPV4",
      "address": "192.168.0.10"
    }
  ],
  "createdAt": "2026-10-22T12:00:00Z",
  "updatedAt": "2026-10-22T12:00:00Z"
}
```

Listing 2: An register response.

2.2 POST /serviceregistry/device-discovery/lookup

Operation: lookup
Input: DeviceLookupRequest
Output: DeviceListResponse

Returns the devices matching all given filters; without filters all devices.

A successful call returns `200 OK` with the output in the body.

```
POST /serviceregistry/device-discovery/lookup HTTP/1.1
Host: serviceregistry:8443
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "deviceNames": [
    "Raspberry1"
  ],
  "addresses": [
    "string"
  ],
  "addressType": "IPV4",
  "metadataRequirementList": [
    {
      "unit": "celsius"
    }
  ]
}
```

Listing 3: An lookup invocation.

```
{
  "entries": [
    {
      "name": "Raspberry1",
      "metadata": {
        "location": "hall-1"
      },
      "addresses": [
        {
          "type": "IPV4",
          "address": "192.168.0.10"
        }
      ],
      "createdAt": "2026-10-22T12:00:00Z",
      "updatedAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```

Listing 4: An lookup response.

2.3 DELETE /serviceregistry/device-discovery/revoke/*{name}*

Operation: revoke
Input: String

Removes the named device. Returns 200 if it was removed, 204 if it did not exist, and 423 Locked if systems are still assigned to it.

Path parameter: *name*.

A successful call returns `200 OK` if the entry was removed, `204 No Content` if there was nothing to remove.

```
DELETE /serviceregistry/device-discovery/revoke/<name> HTTP/1.1
Host: serviceregistry:8443
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 5: An revoke invocation.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.8, which are used to represent things like hashes, identifiers and texts.

3.1 struct Address

A network address with its type.

Object Field	Value Type	Description
"type"	String	Address type: `HOSTNAME`, `IPV4`, `IPV6` or `MAC`.
"address"	String	One network address (host name, IPv4, IPv6 or MAC address).

3.2 struct DeviceListResponse

List of devices.

Object Field	Value Type	Description
"entries"	Array<DeviceResponse>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.3 struct DeviceLookupRequest

Filter for devices; all fields optional, combined with AND.

Object Field	Value Type	Description
"deviceNames"	Array<String>	Device names to match.
"addresses"	Array<String>	Network addresses of the entity.
"addressType"	String	Address type filter: `HOSTNAME`, `IPV4`, `IPV6` or `MAC`.
"metadataRequirementList"	Array<MetadataRequirement>	Metadata requirements; each map is one alternative.

3.4 struct DeviceRequest

A device and its addresses.

Object Field	Value Type	Description
"name"	String	Device name, PascalCase.
"metadata"	Object	Free-form metadata (JSON object).
"addresses"	Array<String>	Network addresses of the entity.

3.5 struct DeviceResponse

A registered device.

Object Field	Value Type	Description
"name"	String	Device name.
"metadata"	Object	Free-form metadata (JSON object).
"addresses"	Array<Address>	Network addresses with their types.
"createdAt"	DateTime	UTC time of creation.
"updatedAt"	DateTime	UTC time of the last change.

3.6 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.7 struct MetadataRequirement

A JSON object of metadata requirements: each key is a metadata path (dot notation), each value the required value or an operator object such as `{"op": "LESS_THAN", "value": 10}`.

3.8 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “deviceDiscovery – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.2.2, 2026, serviceRegistry-v5.2.2-deviceDiscovery-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

deviceDiscovery MQTT/TLS/JSON

Interface Design Description

Service ID: *"deviceDiscovery"*

Abstract
This document describes the MQTT protocol with TLS payload security and JSON payload encoding variant of the `deviceDiscovery` service of the ServiceRegistry system.

Contents

1 Overview **3**

2 Service Operations **4**
2.1 operation register . . . 4
2.2 operation lookup . . . 4
2.3 operation revoke . . . 5

3 Data Models **6**
3.1 struct MqttRequestTemplate . . . 6
3.2 struct MqttResponseTemplate . . . 6
3.3 struct ErrorMessage . . . 6
3.4 struct Address . . . 6
3.5 struct DeviceListResponse . . . 6
3.6 struct DeviceLookupRequest . . . 7
3.7 struct DeviceRequest . . . 7
3.8 struct DeviceResponse . . . 7
3.9 struct MetadataRequirement . . . 7
3.10 Primitives . . . 7

4 References **9**

5 Revision History **10**
5.1 Amendments . . . 10
5.2 Quality Assurance . . . 10

1 Overview

This document describes the `deviceDiscovery` service interface of the ServiceRegistry system over MQTT, which provides registration and lookup of the devices of the local cloud: the physical or virtual hosts, with their network addresses and metadata, on which the systems run.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	MQTT	3.1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `deviceDiscovery` service interface

This document provides the Interface Design Description IDD to the *deviceDiscovery – Service Description* document [5]. The operations, their semantics and the data models are the same as in the HTTP/TLS/JSON IDD [6]; this document specifies the MQTT binding.

The MQTT interface is offered only when the system runs with `mqtt.api.enabled=true`; it is then registered with the interface template `generic_mqtts` (`generic_mqtt` without TLS), the broker address and port (`mqtt.broker.address`, `mqtt.broker.port`) and the base topic `arrowhead/serviceregistry/device-discovery/`. In the Arrowhead-test-cloud-java deployment the MQTT interfaces of the core systems are disabled.

2 Service Operations

The provider subscribes to one topic per operation, the base topic followed by the operation name:

Operation	Request topic
`register`	`arrowhead/serviceregistry/device-discovery/register`
`lookup`	`arrowhead/serviceregistry/device-discovery/lookup`
`revoke`	`arrowhead/serviceregistry/device-discovery/revoke`

A consumer publishes a request envelope (MqttRequestTemplate) to the topic and receives the answer (MqttResponseTemplate) on its own `responseTopic`. The `authentication` field carries the requester's identity according to the provider's authentication policy: `IDENTITY-TOKEN//<token>` (outsourced or internal policy), `SYSTEM//<name>` (declared policy) or the requester's X.509 certificate (certificate policy). The `status` of the response uses the HTTP status code numbers (200, 201, 204, 400, 401, 403, 404, 408, 423, 500, 503); on error the payload is an ErrorMessage.

```
{
  "traceId": "a1b2c3",
  "authentication": "IDENTITY-TOKEN//<token>",
  "responseTopic": "arrowhead/response/TemperatureDisplay/a1b2c3",
  "qosRequirement": 1,
  "params": {},
  "payload": {}
}
```

Listing 1: Request envelope.

```
{
  "status": 200,
  "traceId": "a1b2c3",
  "receiver": "TemperatureDisplay",
  "payload": {}
}
```

Listing 2: Response envelope.

2.1 PUBLISH `arrowhead/serviceregistry/device-discovery/register`

Operation: register
Input: DeviceRequest
Output: DeviceResponse

Registers a device with its addresses and metadata. If a device with the same name and the same data exists, it is returned unchanged (200); a new device is created (201). A device with the same name and different data is refused.

```
{
  "name": "Raspberry1",
  "metadata": {
    "location": "hall-1"
  },
  "addresses": [
    "string"
  ]
}
```

Listing 3: Payload of an register request.

2.2 PUBLISH `arrowhead/serviceregistry/device-discovery/lookup`

Operation: lookup
Input: DeviceLookupRequest
Output: DeviceListResponse

Returns the devices matching all given filters; without filters all devices.

```
{
  "deviceNames": [
    "Raspberry1"
  ],
  "addresses": [
    "string"
  ],
  "addressType": "IPV4",
  "metadataRequirementList": [
    {
      "unit": "celsius"
    }
  ]
}
```

Listing 4: Payload of an lookup request.

2.3 PUBLISH `arrowhead/serviceregistry/device-discovery/revoke`

Operation: revoke
Input: String

Removes the named device. Returns 200 if it was removed, 204 if it did not exist, and 423 Locked if systems are still assigned to it.

```
"TemperatureSensor"
```

Listing 5: Payload of an revoke request.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Each *struct* denotes a JSON Object with the named fields; fields that are not needed may be omitted.

3.1 struct MqttRequestTemplate

The request envelope.

Object Field	Value Type	Description
"traceId"	String	Identifier chosen by the requester, returned in the response.
"authentication"	String	Identity of the requester (see Section 2).
"responseTopic"	String	Topic on which the response is published.
"qosRequirement"	Number	MQTT QoS of the response (0, 1 or 2).
"params"	Object<String>	Optional parameters (the query parameters of the HTTP binding).
"payload"	Object	The operation input.

3.2 struct MqttResponseTemplate

The response envelope.

Object Field	Value Type	Description
"status"	Number	Status code.
"traceId"	String	The trace identifier of the request.
"receiver"	String	Name of the requester.
"payload"	Object	The operation output, or an ErrorMessage.

3.3 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.4 struct Address

A network address with its type.

Object Field	Value Type	Description
"type"	String	Address type: `HOSTNAME`, `IPV4`, `IPV6` or `MAC`.
"address"	String	One network address (host name, IPv4, IPv6 or MAC address).

3.5 struct DeviceListResponse

List of devices.

Object Field	Value Type	Description
"entries"	Array<DeviceResponse>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.6 struct DeviceLookupRequest

Filter for devices; all fields optional, combined with AND.

Object Field	Value Type	Description
"deviceNames"	Array<String>	Device names to match.
"addresses"	Array<String>	Network addresses of the entity.
"addressType"	String	Address type filter: `HOSTNAME`, `IPV4`, `IPV6` or `MAC`.
"metadataRequirement List"	Array<MetadataRequirement>	Metadata requirements; each map is one alternative.

3.7 struct DeviceRequest

A device and its addresses.

Object Field	Value Type	Description
"name"	String	Device name, PascalCase.
"metadata"	Object	Free-form metadata (JSON object).
"addresses"	Array<String>	Network addresses of the entity.

3.8 struct DeviceResponse

A registered device.

Object Field	Value Type	Description
"name"	String	Device name.
"metadata"	Object	Free-form metadata (JSON object).
"addresses"	Array<Address>	Network addresses with their types.
"createdAt"	DateTime	UTC time of creation.
"updatedAt"	DateTime	UTC time of the last change.

3.9 struct MetadataRequirement

A JSON object of metadata requirements: each key is a metadata path (dot notation), each value the required value or an operator object such as `{"op": "LESS_THAN", "value": 10}`.

3.10 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] A. Banks and R. Gupta, “Mqtt version 3.1.1,” OASIS, Standard, 2014.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “deviceDiscovery – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.2.2, 2026, serviceRegistry-v5.2.2-deviceDiscovery-SD.pdf.

[6] ——, “deviceDiscovery HTTP/TLS/JSON – Interface Design Description (ServiceRegistry),” Eclipse Arrowhead, IDD 5.2.2, 2026, serviceRegistry-v5.2.2-deviceDiscovery-HTTP-IDD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

generalManagement HTTP/TLS/JSON

Interface Design Description

Service ID: *"generalManagement"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `generalManagement` service of the ServiceRegistry system.

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation get-log . . . 4
2.2 operation get-config . . . 4

3 Data Models **6**
3.1 struct ErrorMessage . . . 6
3.2 struct KeyValues . . . 6
3.3 struct LogEntry . . . 6
3.4 struct LogEntryListResponse . . . 6
3.5 struct LogRequest . . . 6
3.6 struct Page . . . 7
3.7 Primitives . . . 7

4 References **8**

5 Revision History **9**
5.1 Amendments . . . 9
5.2 Quality Assurance . . . 9

1 Overview

This document describes the `generalManagement` service interface of the ServiceRegistry system, which provides general management of the ServiceRegistry system: reading its log entries and its configuration.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `generalManagement` service interface

This document provides the Interface Design Description IDD to the *generalManagement – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `generalManagement` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https` (`generic_http` when TLS is disabled), access port 8443 and base path `/serviceregistry/general/mgmt`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`get-log`	POST	`/logs`
`get-config`	GET	`/get-config`

2 Service Operations

The interfaces of the `generalManagement` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `generalManagement` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json`. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

2.1 POST /serviceregistry/general/mgmt/logs

Operation: get-log
Input: LogRequest
Output: LogEntryListResponse

Returns a page of the system's log entries matching the filter.

A successful call returns `200 OK` with the output in the body.

```
POST /serviceregistry/general/mgmt/logs HTTP/1.1
Host: serviceregistry:8443
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "from": "2026-10-22T12:00:00Z",
  "to": "2026-10-22T12:00:00Z",
  "severity": "INFO",
  "logger": "eu.arrowhead.serviceregistry.service.ServiceDiscoveryService"
}
```

Listing 1: An get-log invocation.

```
{
  "entries": [
    {
      "logId": "42",
      "entryDate": "2026-10-22T12:00:00Z",
      "logger": "eu.arrowhead.serviceregistry.service.ServiceDiscoveryService",
      "severity": "INFO",
      "message": "Service instance registered: TemperatureSensor|temperatureReading|1.0.0"
    }
  ],
  "count": 1
}
```

Listing 2: An get-log response.

2.2 GET /serviceregistry/general/mgmt/get-config?keys=*{keys}*

Operation: get-config
Output: KeyValues

Returns the effective values of the requested configuration keys. Only the public keys of the system are answered; other keys, including all secrets, are ignored.

Query parameters: *keys* (List<String>, optional). List values are repeated or comma-separated.

A successful call returns `200 OK` with the output in the body.

```
GET /serviceregistry/general/mgmt/get-config?keys=<keys> HTTP/1.1
Host: serviceregistry:8443
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 3: An get-config invocation.

```
{
  "map": {
    "server.port": "8443"
  }
}
```

Listing 4: An get-config response.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.7, which are used to represent things like hashes, identifiers and texts.

3.1 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.2 struct KeyValues

Configuration values of the system.

Object Field	Value Type	Description
"map"	Object<String>	The configuration keys and values; secrets are not returned.

3.3 struct LogEntry

One log entry.

Object Field	Value Type	Description
"logId"	String	Identifier of the log entry.
"entryDate"	DateTime	UTC time of the log entry.
"logger"	String	Logger (Java class) that wrote the entry; as filter, its name.
"severity"	String	Log level, e.g. `INFO`, `WARN`, `ERROR`; as filter, the minimum level.
"message"	String	Text of the log entry or status message of the job.
"exception"	String	Stack trace or exception text of the log entry, if any.

3.4 struct LogEntryListResponse

List of log entries.

Object Field	Value Type	Description
"entries"	Array<LogEntry>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.5 struct LogRequest

Paged filter for the log entries stored in the system's database.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"from"	DateTime	Return entries from this UTC time.
"to"	DateTime	Return entries until this UTC time.
"severity"	String	Minimum log level to return.
"logger"	String	Logger (Java class) that wrote the entry; as filter, its name.

3.6 struct Page

Paging and sorting; all fields optional.

Object Field	Value Type	Description
"page"	Number	Zero-based page number.
"size"	Number	Number of records per page (at most `max.page.size`, default 1000).
"direction"	String	Sort direction, `ASC` or `DESC`.
"sortField"	String	Field to sort by.

3.7 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “generalManagement – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.2.2, 2026, serviceRegistry-v5.2.2-generalManagement-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

generalManagement MQTT/TLS/JSON

Interface Design Description

Service ID: *"generalManagement"*

Abstract
This document describes the MQTT protocol with TLS payload security and JSON payload encoding variant of the `generalManagement` service of the ServiceRegistry system.

Contents

1 Overview **3**

2 Service Operations **4**
2.1 operation get-log . . . 4
2.2 operation get-config . . . 4

3 Data Models **6**
3.1 struct MqttRequestTemplate . . . 6
3.2 struct MqttResponseTemplate . . . 6
3.3 struct ErrorMessage . . . 6
3.4 struct KeyValues . . . 6
3.5 struct LogEntry . . . 6
3.6 struct LogEntryListResponse . . . 7
3.7 struct LogRequest . . . 7
3.8 struct Page . . . 7
3.9 Primitives . . . 7

4 References **9**

5 Revision History **10**
5.1 Amendments . . . 10
5.2 Quality Assurance . . . 10

1 Overview

This document describes the `generalManagement` service interface of the ServiceRegistry system over MQTT, which provides general management of the ServiceRegistry system: reading its log entries and its configuration.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	MQTT	3.1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `generalManagement` service interface

This document provides the Interface Design Description IDD to the *generalManagement – Service Description* document [5]. The operations, their semantics and the data models are the same as in the HTTP/TLS/JSON IDD [6]; this document specifies the MQTT binding.

The MQTT interface is offered only when the system runs with `mqtt.api.enabled=true`; it is then registered with the interface template `generic_mqtts` (`generic_mqtt` without TLS), the broker address and port (`mqtt.broker.address`, `mqtt.broker.port`) and the base topic `arrowhead/serviceregistry/general/management/`. In the Arrowhead-test-cloud-java deployment the MQTT interfaces of the core systems are disabled.

2 Service Operations

The provider subscribes to one topic per operation, the base topic followed by the operation name:

Operation	Request topic
`get-log`	`arrowhead/serviceregistry/general/management/get-log`
`get-config`	`arrowhead/serviceregistry/general/management/get-config`

A consumer publishes a request envelope (MqttRequestTemplate) to the topic and receives the answer (MqttResponseTemplate) on its own `responseTopic`. The `authentication` field carries the requester's identity according to the provider's authentication policy: `IDENTITY-TOKEN//<token>` (outsourced or internal policy), `SYSTEM//<name>` (declared policy) or the requester's X.509 certificate (certificate policy). The `status` of the response uses the HTTP status code numbers (200, 201, 204, 400, 401, 403, 404, 408, 423, 500, 503); on error the payload is an ErrorMessage.

```
{
  "traceId": "a1b2c3",
  "authentication": "IDENTITY-TOKEN//<token>",
  "responseTopic": "arrowhead/response/TemperatureDisplay/a1b2c3",
  "qosRequirement": 1,
  "params": {},
  "payload": {}
}
```

Listing 1: Request envelope.

```
{
  "status": 200,
  "traceId": "a1b2c3",
  "receiver": "TemperatureDisplay",
  "payload": {}
}
```

Listing 2: Response envelope.

2.1 PUBLISH `arrowhead/serviceregistry/general/management/get-log`

Operation: get-log
Input: LogRequest
Output: LogEntryListResponse

Returns a page of the system's log entries matching the filter.

```
{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "from": "2026-10-22T12:00:00Z",
  "to": "2026-10-22T12:00:00Z",
  "severity": "INFO",
  "logger": "eu.arrowhead.serviceregistry.service.ServiceDiscoveryService"
}
```

Listing 3: Payload of an get-log request.

2.2 PUBLISH `arrowhead/serviceregistry/general/management/get-config`

Operation: get-config
Input: Array<String>
Output: KeyValues

Returns the effective values of the requested configuration keys. Only the public keys of the system are answered; other keys, including all secrets, are ignored.

```
[
  "string"
]
```

Listing 4: Payload of an get-config request.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Each *struct* denotes a JSON Object with the named fields; fields that are not needed may be omitted.

3.1 struct MqttRequestTemplate

The request envelope.

Object Field	Value Type	Description
"traceId"	String	Identifier chosen by the requester, returned in the response.
"authentication"	String	Identity of the requester (see Section 2).
"responseTopic"	String	Topic on which the response is published.
"qosRequirement"	Number	MQTT QoS of the response (0, 1 or 2).
"params"	Object<String>	Optional parameters (the query parameters of the HTTP binding).
"payload"	Object	The operation input.

3.2 struct MqttResponseTemplate

The response envelope.

Object Field	Value Type	Description
"status"	Number	Status code.
"traceId"	String	The trace identifier of the request.
"receiver"	String	Name of the requester.
"payload"	Object	The operation output, or an ErrorMessage.

3.3 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.4 struct KeyValues

Configuration values of the system.

Object Field	Value Type	Description
"map"	Object<String>	The configuration keys and values; secrets are not returned.

3.5 struct LogEntry

One log entry.

Object Field	Value Type	Description
"logId"	String	Identifier of the log entry.
"entryDate"	DateTime	UTC time of the log entry.
"logger"	String	Logger (Java class) that wrote the entry; as filter, its name.
"severity"	String	Log level, e.g. `INFO`, `WARN`, `ERROR`; as filter, the minimum level.
"message"	String	Text of the log entry or status message of the job.
"exception"	String	Stack trace or exception text of the log entry, if any.

3.6 struct LogEntryListResponse

List of log entries.

Object Field	Value Type	Description
"entries"	Array<LogEntry>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.7 struct LogRequest

Paged filter for the log entries stored in the system's database.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"from"	DateTime	Return entries from this UTC time.
"to"	DateTime	Return entries until this UTC time.
"severity"	String	Minimum log level to return.
"logger"	String	Logger (Java class) that wrote the entry; as filter, its name.

3.8 struct Page

Paging and sorting; all fields optional.

Object Field	Value Type	Description
"page"	Number	Zero-based page number.
"size"	Number	Number of records per page (at most `max.page.size`, default 1000).
"direction"	String	Sort direction, `ASC` or `DESC`.
"sortField"	String	Field to sort by.

3.9 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] A. Banks and R. Gupta, “Mqtt version 3.1.1,” OASIS, Standard, 2014.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “generalManagement – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.2.2, 2026, serviceRegistry-v5.2.2-generalManagement-SD.pdf.

[6] ——, “generalManagement HTTP/TLS/JSON – Interface Design Description (ServiceRegistry),” Eclipse Arrowhead, IDD 5.2.2, 2026, serviceRegistry-v5.2.2-generalManagement-HTTP-IDD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

serviceDiscovery HTTP/TLS/JSON

Interface Design Description

Service ID: *"serviceDiscovery"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `serviceDiscovery` service of the ServiceRegistry system.

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation register . . . 4
2.2 operation lookup . . . 5
2.3 operation revoke . . . 7

3 Data Models **9**
3.1 struct Address . . . 9
3.2 struct DeviceResponse . . . 9
3.3 struct ErrorMessage . . . 9
3.4 struct MetadataRequirement . . . 9
3.5 struct ServiceDefinitionResponse . . . 9
3.6 struct ServiceInstanceCreateRequest . . . 9
3.7 struct ServiceInstanceInterfaceRequest . . . 10
3.8 struct ServiceInstanceInterfaceResponse . . . 10
3.9 struct ServiceInstanceListResponse . . . 10
3.10 struct ServiceInstanceLookupRequest . . . 10
3.11 struct ServiceInstanceResponse . . . 11
3.12 struct SystemResponse . . . 11
3.13 Primitives . . . 12

4 References **13**

5 Revision History **14**
5.1 Amendments . . . 14
5.2 Quality Assurance . . . 14

1 Overview

This document describes the `serviceDiscovery` service interface of the ServiceRegistry system, which provides registration and lookup of service instances: which provider offers which service definition, in which version, through which interfaces and with which security policy.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `serviceDiscovery` service interface

This document provides the Interface Design Description IDD to the *serviceDiscovery – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `serviceDiscovery` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https` (`generic_http` when TLS is disabled), access port 8443 and base path `/serviceregistry/service-discovery`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`register`	POST	`/register`
`lookup`	POST	`/lookup`
`revoke`	DELETE	`/revoke/{instanceId}`

2 Service Operations

The interfaces of the `serviceDiscovery` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `serviceDiscovery` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json`. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

2.1 POST /serviceregistry/service-discovery/register

Operation: register
Input: ServiceInstanceCreateRequest
Output: ServiceInstanceResponse

Registers a service instance of the requesting system. An existing instance with the same identifier is replaced. Unknown service definitions are created; interfaces are validated against their templates.

A successful call returns `201 Created` with the output in the body.

```
POST /serviceregistry/service-discovery/register HTTP/1.1
Host: serviceregistry:8443
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "serviceDefinitionName": "temperatureReading",
  "version": "1.0.0",
  "expiresAt": "2026-10-22T12:00:00Z",
  "metadata": {
    "unit": "celsius"
  },
  "interfaces": [
    {
      "templateName": "generic_https",
      "protocol": "https",
      "policy": "CERT_AUTH",
      "properties": {
        "accessAddresses": [
          "192.168.0.10"
        ],
        "accessPort": 8080,
        "basePath": "/temperature",
        "operations": {
          "read": {
            "method": "GET",
            "path": "/read"
          }
        }
      }
    }
  ]
}
```

Listing 1: An register invocation.

```
{
  "instanceId": "TemperatureSensor|temperatureReading|1.0.0",
  "provider": {
    "name": "TemperatureSensor",
    "metadata": {
      "location": "hall-1"
    },
    "version": "1.0.0",
    "addresses": [
      {
        "type": "IPV4",
        "address": "192.168.0.10"
      }
    ],
    "device": {
      "name": "Raspberry1",
      "metadata": {
        "location": "hall-1"
      },
      "addresses": [
        {
          "type": "IPV4",
          "address": "192.168.0.10"
        }
      ],
      "createdAt": "2026-10-22T12:00:00Z",
      "updatedAt": "2026-10-22T12:00:00Z"
    },
    "createdAt": "2026-10-22T12:00:00Z",
    "updatedAt": "2026-10-22T12:00:00Z"
  },
  "serviceDefinition": {
    "name": "temperatureReading",
    "createdAt": "2026-10-22T12:00:00Z",
    "updatedAt": "2026-10-22T12:00:00Z"
  },
  "version": "1.0.0",
  "expiresAt": "2026-10-22T12:00:00Z",
  "metadata": {
    "unit": "celsius"
  },
  "interfaces": [
    {
      "templateName": "generic_https",
      "protocol": "https",
      "policy": "CERT_AUTH",
      "properties": {
        "accessAddresses": [
          "192.168.0.10"
        ],
        "accessPort": 8080,
        "basePath": "/temperature",
        "operations": {
          "read": {
            "method": "GET",
            "path": "/read"
          }
        }
      }
    }
  ],
  "createdAt": "2026-10-22T12:00:00Z",
  "updatedAt": "2026-10-22T12:00:00Z"
}
```


Listing 2: An register response.

2.2 POST /serviceregistry/service-discovery/lookup?verbose={verbose}

Operation: lookup
Input: ServiceInstanceLookupRequest
Output: ServiceInstanceListResponse

Returns the service instances matching all given filters. Under the restricted discovery policy, a requester without direct access only receives instances whose service has the `unrestrictedDiscovery` metadata (the core services). With `verbose=true` the provider and device details are included.

Query parameters: *verbose* (boolean, optional, default `false`). List values are repeated or comma-separated.

A successful call returns `200 OK` with the output in the body.

```
POST /serviceregistry/service-discovery/lookup?verbose=<verbose> HTTP/1.1
Host: serviceregistry:8443
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "instanceIds": [
    "TemperatureSensor|temperatureReading|1.0.0"
  ],
  "providerNames": [
    "TemperatureSensor"
  ],
  "serviceDefinitionNames": [
    "temperatureReading"
  ],
  "versions": [
    "1.0.0"
  ],
  "alivesAt": "2026-10-22T12:00:00Z",
  "metadataRequirementsList": [
    {
      "unit": "celsius"
    }
  ],
  "addressTypes": [
    "IPV4"
  ],
  "interfaceTemplateNames": [
    "string"
  ],
  "interfacePropertyRequirementsList": [
    {
      "unit": "celsius"
    }
  ],
  "policies": [
    "string"
  ]
}
```

Listing 3: An lookup invocation.

```
{
  "entries": [
    {
      "instanceId": "TemperatureSensor|temperatureReading|1.0.0",
      "provider": {
        "name": "TemperatureSensor",
        "metadata": {
          "location": "hall-1"
        },
        "version": "1.0.0",
```

```
        "addresses": [
          {}
        ],
        "device": {
          "name": "Raspberry1",
          "metadata": {
            "location": "hall-1"
          },
          "addresses": [
            {}
          ],
          "createdAt": "2026-10-22T12:00:00Z",
          "updatedAt": "2026-10-22T12:00:00Z"
        },
        "createdAt": "2026-10-22T12:00:00Z",
        "updatedAt": "2026-10-22T12:00:00Z"
      },
      "serviceDefinition": {
        "name": "temperatureReading",
        "createdAt": "2026-10-22T12:00:00Z",
        "updatedAt": "2026-10-22T12:00:00Z"
      },
      "version": "1.0.0",
      "expiresAt": "2026-10-22T12:00:00Z",
      "metadata": {
        "unit": "celsius"
      },
      "interfaces": [
        {
          "templateName": "generic_https",
          "protocol": "https",
          "policy": "CERT_AUTH",
          "properties": {
            "accessAddresses": [
              "192.168.0.10"
            ],
            "accessPort": 8080,
            "basePath": "/temperature",
            "operations": {
              "read": {
                "method": "GET",
                "path": "/read"
              }
            }
          }
        }
      ],
      "createdAt": "2026-10-22T12:00:00Z",
      "updatedAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```


Listing 4: An lookup response.

2.3 DELETE /serviceregistry/service-discovery/revoke/*{instanceId}*

Operation: revoke
Input: String

Removes a service instance of the requesting system. Returns 200 if removed, 204 if it did not exist.

Path parameter: *instanceId*.

A successful call returns `200 OK` if the entry was removed, `204 No Content` if there was nothing to remove.

```
DELETE /serviceregistry/service-discovery/revoke/<instanceId> HTTP/1.1
Host: serviceregistry:8443
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 5: An revoke invocation.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.13, which are used to represent things like hashes, identifiers and texts.

3.1 struct Address

A network address with its type.

Object Field	Value Type	Description
"type"	String	Address type: `HOSTNAME`, `IPV4`, `IPV6` or `MAC`.
"address"	String	One network address (host name, IPv4, IPv6 or MAC address).

3.2 struct DeviceResponse

A registered device.

Object Field	Value Type	Description
"name"	String	Device name.
"metadata"	Object	Free-form metadata (JSON object).
"addresses"	Array<Address>	Network addresses with their types.
"createdAt"	DateTime	UTC time of creation.
"updatedAt"	DateTime	UTC time of the last change.

3.3 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.4 struct MetadataRequirement

A JSON object of metadata requirements: each key is a metadata path (dot notation), each value the required value or an operator object such as `{"op": "LESS_THAN", "value": 10}`.

3.5 struct ServiceDefinitionResponse

A service definition.

Object Field	Value Type	Description
"name"	String	Service definition name.
"createdAt"	DateTime	UTC time of creation.
"updatedAt"	DateTime	UTC time of the last change.

3.6 struct ServiceInstanceCreateRequest

Registers a service instance of the requesting system.

Object Field	Value Type	Description
"serviceDefinitionName"	String	Service definition name, camelCase (e.g. `temperatureReading`).
"version"	Version	Version of the service instance (default `1.0.0`).
"expiresAt"	DateTime	UTC expiry time; no value means no expiry.
"metadata"	Object	Free-form metadata (JSON object).
"interfaces"	Array<ServiceInstanceInterface Request>	Service interfaces offered by the instance.

3.7 struct ServiceInstanceInterfaceRequest

One interface of a service instance.

Object Field	Value Type	Description
"templateName"	String	Name of the interface template, snake_case (e.g. `generic_https`).
"protocol"	String	Protocol of the interface, e.g. `http`, `https`, `mqtt`, `mqtts`.
"policy"	String	Security policy of the interface (`NONE`, `CERT_AUTH`, token policies).
"properties"	Object	Interface properties defined by the template, e.g. `accessAddresses`, `accessPort`, `basePath`, `operations`.

3.8 struct ServiceInstanceInterfaceResponse

One interface of a service instance.

Object Field	Value Type	Description
"templateName"	String	Name of the interface template, snake_case (e.g. `generic_https`).
"protocol"	String	Protocol of the interface, e.g. `http`, `https`, `mqtt`, `mqtts`.
"policy"	String	Security policy of the interface: `NONE`, `CERT_AUTH` or one of the token policies.
"properties"	Object	Interface properties defined by the template, e.g. `accessAddresses`, `accessPort`, `basePath`, `operations`.

3.9 struct ServiceInstanceListResponse

List of service instances.

Object Field	Value Type	Description
"entries"	Array<ServiceInstanceResponse>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.10 struct ServiceInstanceLookupRequest

Filter for service instances; all fields optional, combined with AND.

Object Field	Value Type	Description
"instanceIds"	Array<String>	Instance identifiers to match.
"providerNames"	Array<String>	Provider system names to match.
"serviceDefinition Names"	Array<String>	Service definition names.
"versions"	Array<Version>	Service versions to match.
"alivesAt"	DateTime	Only instances that are still registered (not expired) at this UTC time are matched.
"metadataRequirements List"	Array<MetadataRequirement>	Requirements on the service instance metadata; each map is one alternative.
"addressTypes"	Array<String>	Address types the interfaces must offer (`HOSTNAME`, `IPV4`, `IPV6`, `MAC`).
"interfaceTemplate Names"	Array<String>	Accepted interface templates, e.g. `generic_https`.
"interfaceProperty RequirementsList"	Array<MetadataRequirement>	Requirements on the interface properties; each map is one alternative.
"policies"	Array<String>	Interface security policies to match, e.g. `CERT_AUTH`.

3.11 struct ServiceInstanceResponse

A registered service instance.

Object Field	Value Type	Description
"instanceId"	String	Service instance identifier `<System>|<serviceDefinition>|<version>`, or policy instance identifier.
"provider"	SystemResponse	The provider system.
"serviceDefinition"	ServiceDefinitionResponse	The service definition.
"version"	Version	Version of the service instance.
"expiresAt"	DateTime	UTC expiry time; no value means no expiry.
"metadata"	Object	Free-form metadata (JSON object).
"interfaces"	Array<ServiceInstanceInterface Response>	Service interfaces offered by the instance.
"createdAt"	DateTime	UTC time of creation.
"updatedAt"	DateTime	UTC time of the last change.

3.12 struct SystemResponse

A registered system.

Object Field	Value Type	Description
"name"	String	System name.
"metadata"	Object	Free-form metadata (JSON object).
"version"	Version	Version of the system.
"addresses"	Array<Address>	Network addresses with their types.
"device"	DeviceResponse	The device hosting the system, if registered.
"createdAt"	DateTime	UTC time of creation.
"updatedAt"	DateTime	UTC time of the last change.

3.13 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.
Version	JSON String, semantic version `major.minor.patch`.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “serviceDiscovery – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.2.2, 2026, serviceRegistry-v5.2.2-serviceDiscovery-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

serviceDiscovery MQTT/TLS/JSON

Interface Design Description

Service ID: *"serviceDiscovery"*

Abstract
This document describes the MQTT protocol with TLS payload security and JSON payload encoding variant of the `serviceDiscovery` service of the ServiceRegistry system.

Contents

1 Overview **3**

2 Service Operations **4**
2.1 operation register . . . 4
2.2 operation lookup . . . 5
2.3 operation revoke . . . 6

3 Data Models **7**
3.1 struct MqttRequestTemplate . . . 7
3.2 struct MqttResponseTemplate . . . 7
3.3 struct ErrorMessage . . . 7
3.4 struct Address . . . 7
3.5 struct DeviceResponse . . . 7
3.6 struct MetadataRequirement . . . 8
3.7 struct ServiceDefinitionResponse . . . 8
3.8 struct ServiceInstanceCreateRequest . . . 8
3.9 struct ServiceInstanceInterfaceRequest . . . 8
3.10 struct ServiceInstanceInterfaceResponse . . . 8
3.11 struct ServiceInstanceListResponse . . . 9
3.12 struct ServiceInstanceLookupRequest . . . 9
3.13 struct ServiceInstanceResponse . . . 9
3.14 struct SystemResponse . . . 10
3.15 Primitives . . . 10

4 References **11**

5 Revision History **12**
5.1 Amendments . . . 12
5.2 Quality Assurance . . . 12

1 Overview

This document describes the `serviceDiscovery` service interface of the ServiceRegistry system over MQTT, which provides registration and lookup of service instances: which provider offers which service definition, in which version, through which interfaces and with which security policy.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	MQTT	3.1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `serviceDiscovery` service interface

This document provides the Interface Design Description IDD to the *serviceDiscovery – Service Description* document [5]. The operations, their semantics and the data models are the same as in the HTTP/TLS/JSON IDD [6]; this document specifies the MQTT binding.

The MQTT interface is offered only when the system runs with `mqtt.api.enabled=true`; it is then registered with the interface template `generic_mqtts` (`generic_mqtt` without TLS), the broker address and port (`mqtt.broker.address`, `mqtt.broker.port`) and the base topic `arrowhead/serviceregistry/service-discovery/`. In the Arrowhead-test-cloud-java deployment the MQTT interfaces of the core systems are disabled.

2 Service Operations

The provider subscribes to one topic per operation, the base topic followed by the operation name:

Operation	Request topic
`register`	`arrowhead/serviceregistry/service-discovery/register`
`lookup`	`arrowhead/serviceregistry/service-discovery/lookup`
`revoke`	`arrowhead/serviceregistry/service-discovery/revoke`

A consumer publishes a request envelope (MqttRequestTemplate) to the topic and receives the answer (MqttResponseTemplate) on its own `responseTopic`. The `authentication` field carries the requester's identity according to the provider's authentication policy: `IDENTITY-TOKEN//<token>` (outsourced or internal policy), `SYSTEM//<name>` (declared policy) or the requester's X.509 certificate (certificate policy). The `status` of the response uses the HTTP status code numbers (200, 201, 204, 400, 401, 403, 404, 408, 423, 500, 503); on error the payload is an ErrorMessage.

```
{
  "traceId": "a1b2c3",
  "authentication": "IDENTITY-TOKEN//<token>",
  "responseTopic": "arrowhead/response/TemperatureDisplay/a1b2c3",
  "qosRequirement": 1,
  "params": {},
  "payload": {}
}
```

Listing 1: Request envelope.

```
{
  "status": 200,
  "traceId": "a1b2c3",
  "receiver": "TemperatureDisplay",
  "payload": {}
}
```

Listing 2: Response envelope.

2.1 PUBLISH `arrowhead/serviceregistry/service-discovery/register`

Operation: register
Input: ServiceInstanceCreateRequest
Output: ServiceInstanceResponse

Registers a service instance of the requesting system. An existing instance with the same identifier is replaced. Unknown service definitions are created; interfaces are validated against their templates.

```
{
  "serviceDefinitionName": "temperatureReading",
  "version": "1.0.0",
  "expiresAt": "2026-10-22T12:00:00Z",
  "metadata": {
    "unit": "celsius"
  },
  "interfaces": [
    {
      "templateName": "generic_https",
      "protocol": "https",
      "policy": "CERT_AUTH",
      "properties": {
```

```
        "accessAddresses": [
          "192.168.0.10"
        ],
        "accessPort": 8080,
        "basePath": "/temperature",
        "operations": {
          "read": {
            "method": "GET",
            "path": "/read"
          }
        }
      }
    }
  ]
}
```

Listing 3: Payload of an register request.

2.2 PUBLISH `arrowhead/serviceregistry/service-discovery/lookup`

Operation: lookup
Input: ServiceInstanceLookupRequest
Output: ServiceInstanceListResponse

Returns the service instances matching all given filters. Under the restricted discovery policy, a requester without direct access only receives instances whose service has the `unrestrictedDiscovery` metadata (the core services). With `verbose=true` the provider and device details are included.

Optional entries of `params`: `verbose`.

```
{
  "instanceIds": [
    "TemperatureSensor|temperatureReading|1.0.0"
  ],
  "providerNames": [
    "TemperatureSensor"
  ],
  "serviceDefinitionNames": [
    "temperatureReading"
  ],
  "versions": [
    "1.0.0"
  ],
  "alivesAt": "2026-10-22T12:00:00Z",
  "metadataRequirementsList": [
    {
      "unit": "celsius"
    }
  ],
  "addressTypes": [
    "IPV4"
  ],
  "interfaceTemplateNames": [
    "string"
  ],
  "interfacePropertyRequirementsList": [
    {
      "unit": "celsius"
    }
  ],
  "policies": [
    "string"
  ]
}
```

Listing 4: Payload of an lookup request.

2.3 PUBLISH `arrowhead/serviceregistry/service-discovery/revoke`

Operation: revoke
Input: String

Removes a service instance of the requesting system. Returns 200 if removed, 204 if it did not exist.

```
1 "TemperatureSensor|temperatureReading|1.0.0"
```

Listing 5: Payload of an revoke request.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Each *struct* denotes a JSON Object with the named fields; fields that are not needed may be omitted.

3.1 struct MqttRequestTemplate

The request envelope.

Object Field	Value Type	Description
"traceId"	String	Identifier chosen by the requester, returned in the response.
"authentication"	String	Identity of the requester (see Section 2).
"responseTopic"	String	Topic on which the response is published.
"qosRequirement"	Number	MQTT QoS of the response (0, 1 or 2).
"params"	Object<String>	Optional parameters (the query parameters of the HTTP binding).
"payload"	Object	The operation input.

3.2 struct MqttResponseTemplate

The response envelope.

Object Field	Value Type	Description
"status"	Number	Status code.
"traceId"	String	The trace identifier of the request.
"receiver"	String	Name of the requester.
"payload"	Object	The operation output, or an ErrorMessage.

3.3 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.4 struct Address

A network address with its type.

Object Field	Value Type	Description
"type"	String	Address type: `HOSTNAME`, `IPV4`, `IPV6` or `MAC`.
"address"	String	One network address (host name, IPv4, IPv6 or MAC address).

3.5 struct DeviceResponse

A registered device.

Object Field	Value Type	Description
"name"	String	Device name.
"metadata"	Object	Free-form metadata (JSON object).
"addresses"	Array<Address>	Network addresses with their types.
"createdAt"	DateTime	UTC time of creation.
"updatedAt"	DateTime	UTC time of the last change.

3.6 struct MetadataRequirement

A JSON object of metadata requirements: each key is a metadata path (dot notation), each value the required value or an operator object such as `{"op": "LESS_THAN", "value": 10}`.

3.7 struct ServiceDefinitionResponse

A service definition.

Object Field	Value Type	Description
"name"	String	Service definition name.
"createdAt"	DateTime	UTC time of creation.
"updatedAt"	DateTime	UTC time of the last change.

3.8 struct ServiceInstanceCreateRequest

Registers a service instance of the requesting system.

Object Field	Value Type	Description
"serviceDefinitionName"	String	Service definition name, camelCase (e.g. `temperatureReading`).
"version"	Version	Version of the service instance (default `1.0.0`).
"expiresAt"	DateTime	UTC expiry time; no value means no expiry.
"metadata"	Object	Free-form metadata (JSON object).
"interfaces"	Array<ServiceInstanceInterface Request>	Service interfaces offered by the instance.

3.9 struct ServiceInstanceInterfaceRequest

One interface of a service instance.

Object Field	Value Type	Description
"templateName"	String	Name of the interface template, snake_case (e.g. `generic_https`).
"protocol"	String	Protocol of the interface, e.g. `http`, `https`, `mqtt`, `mqtts`.
"policy"	String	Security policy of the interface (`NONE`, `CERT_AUTH`, token policies).
"properties"	Object	Interface properties defined by the template, e.g. `accessAddresses`, `accessPort`, `basePath`, `operations`.

3.10 struct ServiceInstanceInterfaceResponse

One interface of a service instance.

Object Field	Value Type	Description
"templateName"	String	Name of the interface template, snake_case (e.g. `generic_https`).
"protocol"	String	Protocol of the interface, e.g. `http`, `https`, `mqtt`, `mqtts`.
"policy"	String	Security policy of the interface: `NONE`, `CERT_AUTH` or one of the token policies.
"properties"	Object	Interface properties defined by the template, e.g. `accessAddresses`, `accessPort`, `basePath`, `operations`.

3.11 struct ServiceInstanceListResponse

List of service instances.

Object Field	Value Type	Description
"entries"	Array<ServiceInstanceResponse>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.12 struct ServiceInstanceLookupRequest

Filter for service instances; all fields optional, combined with AND.

Object Field	Value Type	Description
"instanceIds"	Array<String>	Instance identifiers to match.
"providerNames"	Array<String>	Provider system names to match.
"serviceDefinition Names"	Array<String>	Service definition names.
"versions"	Array<Version>	Service versions to match.
"alivesAt"	DateTime	Only instances that are still registered (not expired) at this UTC time are matched.
"metadataRequirements List"	Array<MetadataRequirement>	Requirements on the service instance metadata; each map is one alternative.
"addressTypes"	Array<String>	Address types the interfaces must offer (`HOSTNAME`, `IPV4`, `IPV6`, `MAC`).
"interfaceTemplate Names"	Array<String>	Accepted interface templates, e.g. `generic_https`.
"interfaceProperty RequirementsList"	Array<MetadataRequirement>	Requirements on the interface properties; each map is one alternative.
"policies"	Array<String>	Interface security policies to match, e.g. `CERT_AUTH`.

3.13 struct ServiceInstanceResponse

A registered service instance.

Object Field	Value Type	Description
"instanceId"	String	Service instance identifier `<System>|<serviceDefinition>|<version>`, or policy instance identifier.
"provider"	SystemResponse	The provider system.
"serviceDefinition"	ServiceDefinitionResponse	The service definition.
"version"	Version	Version of the service instance.
"expiresAt"	DateTime	UTC expiry time; no value means no expiry.
"metadata"	Object	Free-form metadata (JSON object).
"interfaces"	Array<ServiceInstanceInterface Response>	Service interfaces offered by the instance.
"createdAt"	DateTime	UTC time of creation.
"updatedAt"	DateTime	UTC time of the last change.

3.14 struct SystemResponse

A registered system.

Object Field	Value Type	Description
"name"	String	System name.
"metadata"	Object	Free-form metadata (JSON object).
"version"	Version	Version of the system.
"addresses"	Array<Address>	Network addresses with their types.
"device"	DeviceResponse	The device hosting the system, if registered.
"createdAt"	DateTime	UTC time of creation.
"updatedAt"	DateTime	UTC time of the last change.

3.15 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.
Version	JSON String, semantic version `major.minor.patch`.

4 References

[1] A. Banks and R. Gupta, “Mqtt version 3.1.1,” OASIS, Standard, 2014.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “serviceDiscovery – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.2.2, 2026, serviceRegistry-v5.2.2-serviceDiscovery-SD.pdf.

[6] ——, “serviceDiscovery HTTP/TLS/JSON – Interface Design Description (ServiceRegistry),” Eclipse Arrowhead, IDD 5.2.2, 2026, serviceRegistry-v5.2.2-serviceDiscovery-HTTP-IDD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

serviceRegistryManagement HTTP/TLS/JSON

Interface Design Description

Service ID: *"serviceRegistryManagement"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `serviceRegistryManagement` service of the ServiceRegistry system.

Contents

1 Overview **4**
1.1 Interface registration . . . 4

2 Service Operations **6**
2.1 operation device-query . . . 6
2.2 operation device-create . . . 7
2.3 operation device-update . . . 8
2.4 operation device-remove . . . 8
2.5 operation system-query . . . 9
2.6 operation system-create . . . 10
2.7 operation system-update . . . 11
2.8 operation system-remove . . . 12
2.9 operation service-definition-query . . . 12
2.10 operation service-definition-create . . . 13
2.11 operation service-definition-remove . . . 13
2.12 operation service-query . . . 14
2.13 operation service-create . . . 16
2.14 operation service-update . . . 17
2.15 operation service-remove . . . 19
2.16 operation interface-template-query . . . 19
2.17 operation interface-template-create . . . 20
2.18 operation interface-template-remove . . . 21

3 Data Models **22**
3.1 struct Address . . . 22
3.2 struct DeviceListRequest . . . 22
3.3 struct DeviceListResponse . . . 22
3.4 struct DeviceQueryRequest . . . 22
3.5 struct DeviceRequest . . . 22
3.6 struct DeviceResponse . . . 22
3.7 struct ErrorMessage . . . 23
3.8 struct MetadataRequirement . . . 23
3.9 struct Page . . . 23
3.10 struct ServiceDefinitionListRequest . . . 23
3.11 struct ServiceDefinitionListResponse . . . 23
3.12 struct ServiceDefinitionResponse . . . 23
3.13 struct ServiceInstanceCreateListRequest . . . 24
3.14 struct ServiceInstanceInterfaceRequest . . . 24
3.15 struct ServiceInstanceInterfaceResponse . . . 24
3.16 struct ServiceInstanceListResponse . . . 24
3.17 struct ServiceInstanceQueryRequest . . . 24
3.18 struct ServiceInstanceRequest . . . 25
3.19 struct ServiceInstanceResponse . . . 25
3.20 struct ServiceInstanceUpdateListRequest . . . 26
3.21 struct ServiceInstanceUpdateRequest . . . 26
3.22 struct ServiceInterfaceTemplateListRequest . . . 26
3.23 struct ServiceInterfaceTemplateListResponse . . . 26
3.24 struct ServiceInterfaceTemplateProperty . . . 26
3.25 struct ServiceInterfaceTemplateQueryRequest . . . 27
3.26 struct ServiceInterfaceTemplateRequest . . . 27
3.27 struct ServiceInterfaceTemplateResponse . . . 27
3.28 struct SystemListRequest . . . 27
3.29 struct SystemListResponse . . . 27

3.30 struct SystemQueryRequest 28
3.31 struct SystemRequest 28
3.32 struct SystemResponse 28
3.33 Primitives 28

4 References **30**

5 Revision History **31**
5.1 Amendments 31
5.2 Quality Assurance 31

1 Overview

This document describes the `serviceRegistryManagement` service interface of the ServiceRegistry system, which provides operator management of the whole ServiceRegistry content: devices, systems, service definitions, service instances and interface templates.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `serviceRegistryManagement` service interface

This document provides the Interface Design Description IDD to the *serviceRegistryManagement – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `serviceRegistryManagement` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https` (`generic_http` when TLS is disabled), access port 8443 and base path `/serviceregistry/mgmt`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`device-query`	POST	`/devices/query`
`device-create`	POST	`/devices`
`device-update`	PUT	`/devices`
`device-remove`	DELETE	`/devices`
`system-query`	POST	`/systems/query`
`system-create`	POST	`/systems`
`system-update`	PUT	`/systems`
`system-remove`	DELETE	`/systems`
`service-definition-query`	POST	`/service-definitions/query`
`service-definition-create`	POST	`/service-definitions`
`service-definition-remove`	DELETE	`/service-definitions`
`service-query`	POST	`/service-instances/query`
`service-create`	POST	`/service-instances`
`service-update`	PUT	`/service-instances`
`service-remove`	DELETE	`/service-instances`
`interface-template-query`	POST	`/interface-templates/query`
`interface-template-create`	POST	`/interface-templates`

Operation	Method	Path
`interface-template-remove`	DELETE	`/interface-templates`

2 Service Operations

The interfaces of the `serviceRegistryManagement` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `serviceRegistryManagement` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json`. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

2.1 POST /serviceregistry/mgmt/devices/query

Operation: device-query
Input: DeviceQueryRequest
Output: DeviceListResponse

Returns a page of devices matching the filter.

A successful call returns `200 OK` with the output in the body.

```
POST /serviceregistry/mgmt/devices/query HTTP/1.1
Host: serviceregistry:8443
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "deviceNames": [
    "Raspberry1"
  ],
  "addresses": [
    "string"
  ],
  "addressType": "IPV4",
  "metadataRequirementList": [
    {
      "unit": "celsius"
    }
  ]
}
```

Listing 1: An device-query invocation.

```
{
  "entries": [
    {
      "name": "Raspberry1",
      "metadata": {
        "location": "hall-1"
      },
      "addresses": [
```

```
        {
          "type": "IPV4",
          "address": "192.168.0.10"
        }
      ],
      "createdAt": "2026-10-22T12:00:00Z",
      "updatedAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```

Listing 2: An device-query response.

2.2 POST /serviceregistry/mgmt/devices

Operation: device-create
Input: DeviceListRequest
Output: DeviceListResponse

Creates devices.

A successful call returns `200 OK` with the output in the body.

```
POST /serviceregistry/mgmt/devices HTTP/1.1
Host: serviceregistry:8443
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "devices": [
    {
      "name": "Raspberry1",
      "metadata": {
        "location": "hall-1"
      },
      "addresses": [
        "string"
      ]
    }
  ]
}
```

Listing 3: An device-create invocation.

```
{
  "entries": [
    {
      "name": "Raspberry1",
      "metadata": {
        "location": "hall-1"
      },
      "addresses": [
        {
          "type": "IPV4",
          "address": "192.168.0.10"
        }
      ],
      "createdAt": "2026-10-22T12:00:00Z",
      "updatedAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```

Listing 4: An device-create response.

2.3 PUT /serviceregistry/mgmt/devices

Operation: device-update
Input: DeviceListRequest
Output: DeviceListResponse

Updates the metadata and addresses of devices.
A successful call returns `200 OK` with the output in the body.

```
PUT /serviceregistry/mgmt/devices HTTP/1.1
Host: serviceregistry:8443
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "devices": [
    {
      "name": "Raspberry1",
      "metadata": {
        "location": "hall-1"
      },
      "addresses": [
        "string"
      ]
    }
  ]
}
```

Listing 5: An device-update invocation.

```
{
  "entries": [
    {
      "name": "Raspberry1",
      "metadata": {
        "location": "hall-1"
      },
      "addresses": [
        {
          "type": "IPV4",
          "address": "192.168.0.10"
        }
      ],
      "createdAt": "2026-10-22T12:00:00Z",
      "updatedAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```

Listing 6: An device-update response.

2.4 DELETE /serviceregistry/mgmt/devices?names=*{names}*

Operation: device-remove
Input: Array<String>

Removes devices by name.

Query parameters: *names* (List<String>, required). List values are repeated or comma-separated.
A successful call returns `200 OK`.

```
DELETE /serviceregistry/mgmt/devices?names=<names> HTTP/1.1
Host: serviceregistry:8443
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 7: An device-remove invocation.

2.5 POST /serviceregistry/mgmt/systems/query?verbose=*{verbose}*

Operation: system-query
Input: SystemQueryRequest
Output: SystemListResponse

Returns a page of systems matching the filter.
Query parameters: *verbose* (boolean, optional, default `false`). List values are repeated or comma-separated.
A successful call returns `200 OK` with the output in the body.

```
POST /serviceregistry/mgmt/systems/query?verbose=<verbose> HTTP/1.1
Host: serviceregistry:8443
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "systemNames": [
    "TemperatureSensor"
  ],
  "addresses": [
    "string"
  ],
  "addressType": "IPV4",
  "metadataRequirementList": [
    {
      "unit": "celsius"
    }
  ],
  "versions": [
    "1.0.0"
  ],
  "deviceNames": [
    "Raspberry1"
  ]
}
```

Listing 8: An system-query invocation.

```
{
  "entries": [
    {
      "name": "TemperatureSensor",
      "metadata": {
        "location": "hall-1"
      },
      "version": "1.0.0",
```

```
      "addresses": [
        {
          "type": "IPV4",
          "address": "192.168.0.10"
        }
      ],
      "device": {
        "name": "Raspberry1",
        "metadata": {
          "location": "hall-1"
        },
        "addresses": [
          {}
        ],
        "createdAt": "2026-10-22T12:00:00Z",
        "updatedAt": "2026-10-22T12:00:00Z"
      },
      "createdAt": "2026-10-22T12:00:00Z",
      "updatedAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```

Listing 9: An system-query response.

2.6 POST /serviceregistry/mgmt/systems

Operation: system-create
Input: SystemListRequest
Output: SystemListResponse

Creates systems on behalf of others.

A successful call returns `200 OK` with the output in the body.

```
POST /serviceregistry/mgmt/systems HTTP/1.1
Host: serviceregistry:8443
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "systems": [
    {
      "name": "TemperatureSensor",
      "metadata": {
        "location": "hall-1"
      },
      "version": "1.0.0",
      "addresses": [
        "string"
      ],
      "deviceName": "Raspberry1"
    }
  ]
}
```

Listing 10: An system-create invocation.

```
{
  "entries": [
    {
      "name": "TemperatureSensor",
      "metadata": {
        "location": "hall-1"
```

```
      },
      "version": "1.0.0",
      "addresses": [
        {
          "type": "IPV4",
          "address": "192.168.0.10"
        }
      ],
      "device": {
        "name": "Raspberry1",
        "metadata": {
          "location": "hall-1"
        },
        "addresses": [
          {}
        ],
        "createdAt": "2026-10-22T12:00:00Z",
        "updatedAt": "2026-10-22T12:00:00Z"
      },
      "createdAt": "2026-10-22T12:00:00Z",
      "updatedAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```

Listing 11: An system-create response.

2.7 PUT /serviceregistry/mgmt/systems

Operation: system-update
Input: SystemListRequest
Output: SystemListResponse

Updates systems.

A successful call returns `200 OK` with the output in the body.

```
PUT /serviceregistry/mgmt/systems HTTP/1.1
Host: serviceregistry:8443
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "systems": [
    {
      "name": "TemperatureSensor",
      "metadata": {
        "location": "hall-1"
      },
      "version": "1.0.0",
      "addresses": [
        "string"
      ],
      "deviceName": "Raspberry1"
    }
  ]
}
```

Listing 12: An system-update invocation.

```
{
  "entries": [
    {
      "name": "TemperatureSensor",
```

```
      "metadata": {
        "location": "hall-1"
      },
      "version": "1.0.0",
      "addresses": [
        {
          "type": "IPV4",
          "address": "192.168.0.10"
        }
      ],
      "device": {
        "name": "Raspberry1",
        "metadata": {
          "location": "hall-1"
        },
        "addresses": [
          {}
        ],
        "createdAt": "2026-10-22T12:00:00Z",
        "updatedAt": "2026-10-22T12:00:00Z"
      },
      "createdAt": "2026-10-22T12:00:00Z",
      "updatedAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```

Listing 13: An system-update response.

2.8 DELETE /serviceregistry/mgmt/systems?names=*{names}*

Operation: system-remove
Input: Array<String>

Removes systems by name, with their service instances.

Query parameters: *names* (List<String>, required). List values are repeated or comma-separated.

A successful call returns `200 OK`.

```
DELETE /serviceregistry/mgmt/systems?names=<names> HTTP/1.1
Host: serviceregistry:8443
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 14: An system-remove invocation.

2.9 POST /serviceregistry/mgmt/service-definitions/query

Operation: service-definition-query
Input: Page
Output: ServiceDefinitionListResponse

Returns a page of service definitions.

A successful call returns `200 OK` with the output in the body.

```
POST /serviceregistry/mgmt/service-definitions/query HTTP/1.1
Host: serviceregistry:8443
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "page": 0,
  "size": 10,
```

```
  "direction": "ASC",
  "sortField": "createdAt"
}
```

Listing 15: An service-definition-query invocation.

```
{
  "entries": [
    {
      "name": "temperatureReading",
      "createdAt": "2026-10-22T12:00:00Z",
      "updatedAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```

Listing 16: An service-definition-query response.

2.10 POST /serviceregistry/mgmt/service-definitions

Operation: service-definition-create
Input: ServiceDefinitionListRequest
Output: ServiceDefinitionListResponse

Creates service definitions.

A successful call returns `200 OK` with the output in the body.

```
POST /serviceregistry/mgmt/service-definitions HTTP/1.1
Host: serviceregistry:8443
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "serviceDefinitionNames": [
    "temperatureReading"
  ]
}
```

Listing 17: An service-definition-create invocation.

```
{
  "entries": [
    {
      "name": "temperatureReading",
      "createdAt": "2026-10-22T12:00:00Z",
      "updatedAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```

Listing 18: An service-definition-create response.

2.11 DELETE /serviceregistry/mgmt/service-definitions?names=*{names}*

Operation: service-definition-remove
Input: Array<String>

Removes service definitions by name, with their service instances.

Query parameters: *names* (List<String>, required). List values are repeated or comma-separated.
A successful call returns `200 OK`.

```
DELETE /serviceregistry/mgmt/service-definitions?names=<names> HTTP/1.1
Host: serviceregistry:8443
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 19: An service-definition-remove invocation.

2.12 POST /serviceregistry/mgmt/service-instances/query?verbose={verbose}

Operation: service-query
Input: ServiceInstanceQueryRequest
Output: ServiceInstanceListResponse

Returns a page of service instances matching the filter, regardless of the discovery policy.
Query parameters: *verbose* (boolean, optional, default `false`). List values are repeated or comma-separated.
A successful call returns `200 OK` with the output in the body.

```
POST /serviceregistry/mgmt/service-instances/query?verbose=<verbose> HTTP/1.1
Host: serviceregistry:8443
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "instanceIds": [
    "TemperatureSensor|temperatureReading|1.0.0"
  ],
  "providerNames": [
    "TemperatureSensor"
  ],
  "serviceDefinitionNames": [
    "temperatureReading"
  ],
  "versions": [
    "1.0.0"
  ],
  "alivesAt": "2026-10-22T12:00:00Z",
  "metadataRequirementsList": [
    {
      "unit": "celsius"
    }
  ],
  "addressTypes": [
    "IPV4"
  ],
  "interfaceTemplateNames": [
    "string"
  ],
  "interfacePropertyRequirementsList": [
    {
      "unit": "celsius"
    }
  ],
  "policies": [
    "string"
  ]
```

```
}
```

Listing 20: An service-query invocation.

```
{
  "entries": [
    {
      "instanceId": "TemperatureSensor|temperatureReading|1.0.0",
      "provider": {
        "name": "TemperatureSensor",
        "metadata": {
          "location": "hall-1"
        },
        "version": "1.0.0",
        "addresses": [
          {}
        ],
        "device": {
          "name": "Raspberry1",
          "metadata": {
            "location": "hall-1"
          },
          "addresses": [
            {}
          ],
          "createdAt": "2026-10-22T12:00:00Z",
          "updatedAt": "2026-10-22T12:00:00Z"
        },
        "createdAt": "2026-10-22T12:00:00Z",
        "updatedAt": "2026-10-22T12:00:00Z"
      },
      "serviceDefinition": {
        "name": "temperatureReading",
        "createdAt": "2026-10-22T12:00:00Z",
        "updatedAt": "2026-10-22T12:00:00Z"
      },
      "version": "1.0.0",
      "expiresAt": "2026-10-22T12:00:00Z",
      "metadata": {
        "unit": "celsius"
      },
      "interfaces": [
        {
          "templateName": "generic_https",
          "protocol": "https",
          "policy": "CERT_AUTH",
          "properties": {
            "accessAddresses": [
              "192.168.0.10"
            ],
            "accessPort": 8080,
            "basePath": "/temperature",
            "operations": {
              "read": {
                "method": "GET",
                "path": "/read"
              }
            }
          }
        }
      ],
      "createdAt": "2026-10-22T12:00:00Z",
      "updatedAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```

Listing 21: An service-query response.

2.13 POST /serviceregistry/mgmt/service-instances

Operation: service-create
Input: ServiceInstanceCreateListRequest
Output: ServiceInstanceListResponse

Creates service instances on behalf of providers. The reservation of core and support service definitions applies.

A successful call returns `200 OK` with the output in the body.

```
POST /serviceregistry/mgmt/service-instances HTTP/1.1
Host: serviceregistry:8443
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "instances": [
    {
      "systemName": "TemperatureSensor",
      "serviceDefinitionName": "temperatureReading",
      "version": "1.0.0",
      "expiresAt": "2026-10-22T12:00:00Z",
      "metadata": {
        "unit": "celsius"
      },
      "interfaces": [
        {
          "templateName": "generic_https",
          "protocol": "https",
          "policy": "CERT_AUTH",
          "properties": {
            "accessAddresses": [
              "192.168.0.10"
            ],
            "accessPort": 8080,
            "basePath": "/temperature",
            "operations": {
              "read": {
                "method": "GET",
                "path": "/read"
              }
            }
          }
        }
      ]
    }
  ]
}
```

Listing 22: An service-create invocation.

```
{
  "entries": [
    {
      "instanceId": "TemperatureSensor|temperatureReading|1.0.0",
      "provider": {
        "name": "TemperatureSensor",
        "metadata": {
          "location": "hall-1"
        },
```

```
        "version": "1.0.0",
        "addresses": [
          {}
        ],
        "device": {
          "name": "Raspberry1",
          "metadata": {
            "location": "hall-1"
          },
          "addresses": [
            {}
          ],
          "createdAt": "2026-10-22T12:00:00Z",
          "updatedAt": "2026-10-22T12:00:00Z"
        },
        "createdAt": "2026-10-22T12:00:00Z",
        "updatedAt": "2026-10-22T12:00:00Z"
      },
      "serviceDefinition": {
        "name": "temperatureReading",
        "createdAt": "2026-10-22T12:00:00Z",
        "updatedAt": "2026-10-22T12:00:00Z"
      },
      "version": "1.0.0",
      "expiresAt": "2026-10-22T12:00:00Z",
      "metadata": {
        "unit": "celsius"
      },
      "interfaces": [
        {
          "templateName": "generic_https",
          "protocol": "https",
          "policy": "CERT_AUTH",
          "properties": {
            "accessAddresses": [
              "192.168.0.10"
            ],
            "accessPort": 8080,
            "basePath": "/temperature",
            "operations": {
              "read": {
                "method": "GET",
                "path": "/read"
              }
            }
          }
        }
      ],
      "createdAt": "2026-10-22T12:00:00Z",
      "updatedAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```


Listing 23: An service-create response.

2.14 PUT /serviceregistry/mgmt/service-instances

Operation: service-update
Input: ServiceInstanceUpdateListRequest
Output: ServiceInstanceListResponse

Updates the expiry, metadata and interfaces of service instances.

A successful call returns `200 OK` with the output in the body.

```
PUT /serviceregistry/mgmt/service-instances HTTP/1.1
Host: serviceregistry:8443
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "instances": [
    {
      "instanceId": "TemperatureSensor|temperatureReading|1.0.0",
      "expiresAt": "2026-10-22T12:00:00Z",
      "metadata": {
        "unit": "celsius"
      },
      "interfaces": [
        {
          "templateName": "generic_https",
          "protocol": "https",
          "policy": "CERT_AUTH",
          "properties": {
            "accessAddresses": [
              "192.168.0.10"
            ],
            "accessPort": 8080,
            "basePath": "/temperature",
            "operations": {
              "read": {
                "method": "GET",
                "path": "/read"
              }
            }
          }
        }
      ]
    }
  ]
}
```

Listing 24: An service-update invocation.

```
{
  "entries": [
    {
      "instanceId": "TemperatureSensor|temperatureReading|1.0.0",
      "provider": {
        "name": "TemperatureSensor",
        "metadata": {
          "location": "hall-1"
        },
        "version": "1.0.0",
        "addresses": [
          {}
        ],
        "device": {
          "name": "Raspberry1",
          "metadata": {
            "location": "hall-1"
          },
          "addresses": [
            {}
          ],
          "createdAt": "2026-10-22T12:00:00Z",
          "updatedAt": "2026-10-22T12:00:00Z"
        },
        "createdAt": "2026-10-22T12:00:00Z",
        "updatedAt": "2026-10-22T12:00:00Z"
      },
      "serviceDefinition": {
```

```
        "name": "temperatureReading",
        "createdAt": "2026-10-22T12:00:00Z",
        "updatedAt": "2026-10-22T12:00:00Z"
      },
      "version": "1.0.0",
      "expiresAt": "2026-10-22T12:00:00Z",
      "metadata": {
        "unit": "celsius"
      },
      "interfaces": [
        {
          "templateName": "generic_https",
          "protocol": "https",
          "policy": "CERT_AUTH",
          "properties": {
            "accessAddresses": [
              "192.168.0.10"
            ],
            "accessPort": 8080,
            "basePath": "/temperature",
            "operations": {
              "read": {
                "method": "GET",
                "path": "/read"
              }
            }
          }
        }
      ],
      "createdAt": "2026-10-22T12:00:00Z",
      "updatedAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```

Listing 25: An service-update response.

2.15 DELETE /serviceregistry/mgmt/service-instances?serviceInstances={*serviceInstances*}

Operation: service-remove
Input: Array<String>

Removes service instances by identifier.

Query parameters: *serviceInstances* (List<String>, required). List values are repeated or comma-separated.

A successful call returns `200 OK`.

```
DELETE /serviceregistry/mgmt/service-instances?serviceInstances=<serviceInstances> HTTP/1.1
Host: serviceregistry:8443
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 26: An service-remove invocation.

2.16 POST /serviceregistry/mgmt/interface-templates/query

Operation: interface-template-query
Input: ServiceInterfaceTemplateQueryRequest
Output: ServiceInterfaceTemplateListResponse

Returns a page of interface templates.

A successful call returns `200 OK` with the output in the body.

```
POST /serviceregistry/mgmt/interface-templates/query HTTP/1.1
Host: serviceregistry:8443
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "templateNames": [
    "generic_https"
  ],
  "protocols": [
    "https"
  ]
}
```

Listing 27: An interface-template-query invocation.

```
{
  "entries": [
    {
      "name": "generic_https",
      "protocol": "https",
      "propertyRequirements": [
        {
          "name": "accessPort",
          "mandatory": true,
          "validator": "PORT",
          "validatorParams": [
            "string"
          ]
        }
      ],
      "createdAt": "2026-10-22T12:00:00Z",
      "updatedAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```

Listing 28: An interface-template-query response.

2.17 POST /serviceregistry/mgmt/interface-templates

Operation: interface-template-create
Input: ServiceInterfaceTemplateListRequest
Output: ServiceInterfaceTemplateListResponse

Creates interface templates with their property rules.
A successful call returns `200 OK` with the output in the body.

```
POST /serviceregistry/mgmt/interface-templates HTTP/1.1
Host: serviceregistry:8443
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "interfaceTemplates": [
    {
```

```
      "name": "generic_https",
      "protocol": "https",
      "propertyRequirements": [
        {
          "name": "accessPort",
          "mandatory": true,
          "validator": "PORT",
          "validatorParams": [
            "string"
          ]
        }
      ]
    }
  ]
}
```

Listing 29: An interface-template-create invocation.

```
{
  "entries": [
    {
      "name": "generic_https",
      "protocol": "https",
      "propertyRequirements": [
        {
          "name": "accessPort",
          "mandatory": true,
          "validator": "PORT",
          "validatorParams": [
            "string"
          ]
        }
      ],
      "createdAt": "2026-10-22T12:00:00Z",
      "updatedAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```

Listing 30: An interface-template-create response.

2.18 DELETE /serviceregistry/mgmt/interface-templates?names=*{names}*

Operation: interface-template-remove
Input: Array<String>

Removes interface templates by name.
Query parameters: *names* (List<String>, required). List values are repeated or comma-separated.
A successful call returns `200 OK`.

```
DELETE /serviceregistry/mgmt/interface-templates?names=<names> HTTP/1.1
Host: serviceregistry:8443
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 31: An interface-template-remove invocation.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.33, which are used to represent things like hashes, identifiers and texts.

3.1 struct Address

A network address with its type.

Object Field	Value Type	Description
"type"	String	Address type: `HOSTNAME`, `IPV4`, `IPV6` or `MAC`.
"address"	String	One network address (host name, IPv4, IPv6 or MAC address).

3.2 struct DeviceListRequest

Devices to create or update.

Object Field	Value Type	Description
"devices"	Array<DeviceRequest>	Devices to create or update.

3.3 struct DeviceListResponse

List of devices.

Object Field	Value Type	Description
"entries"	Array<DeviceResponse>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.4 struct DeviceQueryRequest

Paged filter for devices.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"deviceNames"	Array<String>	Device names to match.
"addresses"	Array<String>	Network addresses of the entity.
"addressType"	String	Address type filter: `HOSTNAME`, `IPV4`, `IPV6` or `MAC`.
"metadataRequirementList"	Array<MetadataRequirement>	Metadata requirements; each map is one alternative.

3.5 struct DeviceRequest

A device and its addresses.

Object Field	Value Type	Description
"name"	String	Device name, PascalCase.
"metadata"	Object	Free-form metadata (JSON object).
"addresses"	Array<String>	Network addresses of the entity.

3.6 struct DeviceResponse

A registered device.

Object Field	Value Type	Description
"name"	String	Device name.
"metadata"	Object	Free-form metadata (JSON object).
"addresses"	Array<Address>	Network addresses with their types.
"createdAt"	DateTime	UTC time of creation.
"updatedAt"	DateTime	UTC time of the last change.

3.7 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.8 struct MetadataRequirement

A JSON object of metadata requirements: each key is a metadata path (dot notation), each value the required value or an operator object such as `{"op": "LESS_THAN", "value": 10}`.

3.9 struct Page

Paging and sorting; all fields optional.

Object Field	Value Type	Description
"page"	Number	Zero-based page number.
"size"	Number	Number of records per page (at most `max.page.size`, default 1000).
"direction"	String	Sort direction, `ASC` or `DESC`.
"sortField"	String	Field to sort by.

3.10 struct ServiceDefinitionListRequest

Service definitions to create.

Object Field	Value Type	Description
"serviceDefinition Names"	Array<String>	Service definition names.

3.11 struct ServiceDefinitionListResponse

List of service definitions.

Object Field	Value Type	Description
"entries"	Array<ServiceDefinitionResponse>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.12 struct ServiceDefinitionResponse

A service definition.

Object Field	Value Type	Description
"name"	String	Service definition name.
"createdAt"	DateTime	UTC time of creation.
"updatedAt"	DateTime	UTC time of the last change.

3.13 struct ServiceInstanceCreateListRequest

Service instances to create on behalf of providers.

Object Field	Value Type	Description
"instances"	Array<ServiceInstanceRequest>	Service instances to create or update.

3.14 struct ServiceInstanceInterfaceRequest

One interface of a service instance.

Object Field	Value Type	Description
"templateName"	String	Name of the interface template, snake_case (e.g. `generic_https`).
"protocol"	String	Protocol of the interface, e.g. `http`, `https`, `mqtt`, `mqtts`.
"policy"	String	Security policy of the interface (`NONE`, `CERT_AUTH`, token policies).
"properties"	Object	Interface properties defined by the template, e.g. `accessAddresses`, `accessPort`, `basePath`, `operations`.

3.15 struct ServiceInstanceInterfaceResponse

One interface of a service instance.

Object Field	Value Type	Description
"templateName"	String	Name of the interface template, snake_case (e.g. `generic_https`).
"protocol"	String	Protocol of the interface, e.g. `http`, `https`, `mqtt`, `mqtts`.
"policy"	String	Security policy of the interface: `NONE`, `CERT_AUTH` or one of the token policies.
"properties"	Object	Interface properties defined by the template, e.g. `accessAddresses`, `accessPort`, `basePath`, `operations`.

3.16 struct ServiceInstanceListResponse

List of service instances.

Object Field	Value Type	Description
"entries"	Array<ServiceInstanceResponse>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.17 struct ServiceInstanceQueryRequest

Paged filter for service instances.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"instanceIds"	Array<String>	Instance identifiers to match.
"providerNames"	Array<String>	Provider system names to match.
"serviceDefinition Names"	Array<String>	Service definition names.
"versions"	Array<Version>	Service versions to match.
"alivesAt"	DateTime	Only instances that are still registered (not expired) at this UTC time are matched.
"metadataRequirements List"	Array<MetadataRequirement>	Requirements on the service instance metadata; each map is one alternative.
"addressTypes"	Array<String>	Address types the interfaces must offer (`HOSTNAME`, `IPV4`, `IPV6`, `MAC`).
"interfaceTemplate Names"	Array<String>	Accepted interface templates, e.g. `generic_https`.
"interfaceProperty RequirementsList"	Array<MetadataRequirement>	Requirements on the interface properties; each map is one alternative.
"policies"	Array<String>	Interface security policies to match, e.g. `CERT_AUTH`.

3.18 struct ServiceInstanceRequest

A service instance and its provider.

Object Field	Value Type	Description
"systemName"	String	Name of the provider system.
"serviceDefinitionName"	String	Service definition name, camelCase (e.g. `temperatureReading`).
"version"	Version	Version of the service instance (default `1.0.0`).
"expiresAt"	DateTime	UTC expiry time; no value means no expiry.
"metadata"	Object	Free-form metadata (JSON object).
"interfaces"	Array<ServiceInstanceInterface Request>	Service interfaces offered by the instance.

3.19 struct ServiceInstanceResponse

A registered service instance.

Object Field	Value Type	Description
"instanceId"	String	Service instance identifier `<System>|<serviceDefinition>|<version>`, or policy instance identifier.
"provider"	SystemResponse	The provider system.
"serviceDefinition"	ServiceDefinitionResponse	The service definition.
"version"	Version	Version of the service instance.
"expiresAt"	DateTime	UTC expiry time; no value means no expiry.
"metadata"	Object	Free-form metadata (JSON object).
"interfaces"	Array<ServiceInstanceInterface Response>	Service interfaces offered by the instance.
"createdAt"	DateTime	UTC time of creation.
"updatedAt"	DateTime	UTC time of the last change.

3.20 struct ServiceInstanceUpdateListRequest

Service instances to update.

Object Field	Value Type	Description
"instances"	Array<ServiceInstanceUpdate Request>	Service instances to create or update.

3.21 struct ServiceInstanceUpdateRequest

Changes to one service instance.

Object Field	Value Type	Description
"instanceId"	String	Instance to update.
"expiresAt"	DateTime	UTC expiry time; no value means no expiry.
"metadata"	Object	Free-form metadata (JSON object).
"interfaces"	Array<ServiceInstanceInterface Request>	Service interfaces offered by the instance.

3.22 struct ServiceInterfaceTemplateListRequest

Interface templates to create.

Object Field	Value Type	Description
"interfaceTemplates"	Array<ServiceInterfaceTemplate Request>	Interface templates to create.

3.23 struct ServiceInterfaceTemplateListResponse

List of interface templates.

Object Field	Value Type	Description
"entries"	Array<ServiceInterfaceTemplate Response>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.24 struct ServiceInterfaceTemplateProperty

Rule for one interface property.

Object Field	Value Type	Description
"name"	String	Name of the interface property.
"mandatory"	Boolean	Whether the property must be present in every interface of this template.
"validator"	String	Validator of the value, e.g. `PORT`, `NOT_EMPTY_ADDRESS_LIST`, `HTTP_OPERATIONS`.
"validatorParams"	Array<String>	Parameters of the validator.

3.25 struct ServiceInterfaceTemplateQueryRequest

Paged filter for interface templates.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"templateNames"	Array<String>	Interface template names to match.
"protocols"	Array<String>	Protocols to match.

3.26 struct ServiceInterfaceTemplateRequest

An interface template.

Object Field	Value Type	Description
"name"	String	Template name, snake_case.
"protocol"	String	Protocol of interfaces of this template.
"propertyRequirements"	Array<ServiceInterfaceTemplateProperty>	Property rules of the template.

3.27 struct ServiceInterfaceTemplateResponse

A stored interface template.

Object Field	Value Type	Description
"name"	String	Template name.
"protocol"	String	Protocol of the interface, e.g. `http`, `https`, `mqtt`, `mqtts`.
"propertyRequirements"	Array<ServiceInterfaceTemplateProperty>	Property rules of the template.
"createdAt"	DateTime	UTC time of creation.
"updatedAt"	DateTime	UTC time of the last change.

3.28 struct SystemListRequest

Systems to create or update.

Object Field	Value Type	Description
"systems"	Array<SystemRequest>	Systems to create or update.

3.29 struct SystemListResponse

List of systems.

Object Field	Value Type	Description
"entries"	Array<SystemResponse>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.30 struct SystemQueryRequest

Paged filter for systems.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"systemNames"	Array<String>	System names to match.
"addresses"	Array<String>	Network addresses of the entity.
"addressType"	String	Address type filter: `HOSTNAME`, `IPV4`, `IPV6` or `MAC`.
"metadataRequirementList"	Array<MetadataRequirement>	Metadata requirements; each map is one alternative.
"versions"	Array<Version>	System versions to match.
"deviceNames"	Array<String>	Device names to match.

3.31 struct SystemRequest

A system on behalf of which an operator registers.

Object Field	Value Type	Description
"name"	String	System name, PascalCase.
"metadata"	Object	Free-form metadata (JSON object).
"version"	Version	Version of the system.
"addresses"	Array<String>	Network addresses of the entity.
"deviceName"	String	Name of the device hosting the system.

3.32 struct SystemResponse

A registered system.

Object Field	Value Type	Description
"name"	String	System name.
"metadata"	Object	Free-form metadata (JSON object).
"version"	Version	Version of the system.
"addresses"	Array<Address>	Network addresses with their types.
"device"	DeviceResponse	The device hosting the system, if registered.
"createdAt"	DateTime	UTC time of creation.
"updatedAt"	DateTime	UTC time of the last change.

3.33 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns

syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
Boolean	JSON Boolean, `true` or `false`.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.
Version	JSON String, semantic version `major.minor.patch`.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “serviceRegistryManagement – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.2.2, 2026, serviceRegistry-v5.2.2-serviceRegistryManagement-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

serviceRegistryManagement MQTT/TLS/JSON

Interface Design Description

Service ID: *"serviceRegistryManagement"*

Abstract
This document describes the MQTT protocol with TLS payload security and JSON payload encoding variant of the `serviceRegistryManagement` service of the ServiceRegistry system.

Contents

1 Overview **4**

2 Service Operations **5**
2.1 operation device-query . . . 6
2.2 operation device-create . . . 6
2.3 operation device-update . . . 6
2.4 operation device-remove . . . 7
2.5 operation system-query . . . 7
2.6 operation system-create . . . 8
2.7 operation system-update . . . 8
2.8 operation system-remove . . . 9
2.9 operation service-definition-query . . . 9
2.10 operation service-definition-create . . . 9
2.11 operation service-definition-remove . . . 9
2.12 operation service-query . . . 10
2.13 operation service-create . . . 11
2.14 operation service-update . . . 11
2.15 operation service-remove . . . 12
2.16 operation interface-template-query . . . 12
2.17 operation interface-template-create . . . 12
2.18 operation interface-template-remove . . . 13

3 Data Models **14**
3.1 struct MqttRequestTemplate . . . 14
3.2 struct MqttResponseTemplate . . . 14
3.3 struct ErrorMessage . . . 14
3.4 struct Address . . . 14
3.5 struct DeviceListRequest . . . 14
3.6 struct DeviceListResponse . . . 15
3.7 struct DeviceQueryRequest . . . 15
3.8 struct DeviceRequest . . . 15
3.9 struct DeviceResponse . . . 15
3.10 struct MetadataRequirement . . . 15
3.11 struct Page . . . 15
3.12 struct ServiceDefinitionListRequest . . . 16
3.13 struct ServiceDefinitionListResponse . . . 16
3.14 struct ServiceDefinitionResponse . . . 16
3.15 struct ServiceInstanceCreateListRequest . . . 16
3.16 struct ServiceInstanceInterfaceRequest . . . 16
3.17 struct ServiceInstanceInterfaceResponse . . . 17
3.18 struct ServiceInstanceListResponse . . . 17
3.19 struct ServiceInstanceQueryRequest . . . 17
3.20 struct ServiceInstanceRequest . . . 18
3.21 struct ServiceInstanceResponse . . . 18
3.22 struct ServiceInstanceUpdateListRequest . . . 19
3.23 struct ServiceInstanceUpdateRequest . . . 19
3.24 struct ServiceInterfaceTemplateListRequest . . . 19
3.25 struct ServiceInterfaceTemplateListResponse . . . 19
3.26 struct ServiceInterfaceTemplateProperty . . . 19
3.27 struct ServiceInterfaceTemplateQueryRequest . . . 20
3.28 struct ServiceInterfaceTemplateRequest . . . 20
3.29 struct ServiceInterfaceTemplateResponse . . . 20
3.30 struct SystemListRequest . . . 20

3.31 struct SystemListResponse 20
3.32 struct SystemQueryRequest 21
3.33 struct SystemRequest 21
3.34 struct SystemResponse 21
3.35 Primitives 21

4 References **23**

5 Revision History **24**
5.1 Amendments 24
5.2 Quality Assurance 24

1 Overview

This document describes the `serviceRegistryManagement` service interface of the ServiceRegistry system over MQTT, which provides operator management of the whole ServiceRegistry content: devices, systems, service definitions, service instances and interface templates.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	MQTT	3.1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `serviceRegistryManagement` service interface

This document provides the Interface Design Description IDD to the *serviceRegistryManagement – Service Description* document [5]. The operations, their semantics and the data models are the same as in the HTTP/TLS/JSON IDD [6]; this document specifies the MQTT binding.

The MQTT interface is offered only when the system runs with `mqtt.api.enabled=true`; it is then registered with the interface template `generic_mqtts` (`generic_mqtt` without TLS), the broker address and port (`mqtt.broker.address`, `mqtt.broker.port`) and the base topic `arrowhead/serviceregistry/management/`. In the Arrowhead-test-cloud-java deployment the MQTT interfaces of the core systems are disabled.

2 Service Operations

The provider subscribes to one topic per operation, the base topic followed by the operation name:

Operation	Request topic
device-query	arrowhead/serviceregistry/management/device-query
device-create	arrowhead/serviceregistry/management/device-create
device-update	arrowhead/serviceregistry/management/device-update
device-remove	arrowhead/serviceregistry/management/device-remove
system-query	arrowhead/serviceregistry/management/system-query
system-create	arrowhead/serviceregistry/management/system-create
system-update	arrowhead/serviceregistry/management/system-update
system-remove	arrowhead/serviceregistry/management/system-remove
service-definition-query	arrowhead/serviceregistry/management/service-definition-query
service-definition-create	arrowhead/serviceregistry/management/service-definition-create
service-definition-remove	arrowhead/serviceregistry/management/service-definition-remove
service-query	arrowhead/serviceregistry/management/service-query
service-create	arrowhead/serviceregistry/management/service-create
service-update	arrowhead/serviceregistry/management/service-update
service-remove	arrowhead/serviceregistry/management/service-remove
interface-template-query	arrowhead/serviceregistry/management/interface-template-query
interface-template-create	arrowhead/serviceregistry/management/interface-template-create
interface-template-remove	arrowhead/serviceregistry/management/interface-template-remove

A consumer publishes a request envelope (MqttRequestTemplate) to the topic and receives the answer (MqttResponseTemplate) on its own `responseTopic`. The `authentication` field carries the requester's identity according to the provider's authentication policy: `IDENTITY-TOKEN//<token>` (outsourced or internal policy), `SYSTEM//<name>` (declared policy) or the requester's X.509 certificate (certificate policy). The `status` of the response uses the HTTP status code numbers (200, 201, 204, 400, 401, 403, 404, 408, 423, 500, 503); on error the payload is an ErrorMessage.

```
{
  "traceId": "a1b2c3",
  "authentication": "IDENTITY-TOKEN//<token>",
  "responseTopic": "arrowhead/response/TemperatureDisplay/a1b2c3",
  "qosRequirement": 1,
  "params": {},
  "payload": {}
}
```

Listing 1: Request envelope.

```
{
  "status": 200,
  "traceId": "a1b2c3",
  "receiver": "TemperatureDisplay",
  "payload": {}
```

```
}
```

Listing 2: Response envelope.

2.1 PUBLISH arrowhead/serviceregistry/management/device-query

Operation: device-query
Input: DeviceQueryRequest
Output: DeviceListResponse

Returns a page of devices matching the filter.

```
{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "deviceNames": [
    "Raspberry1"
  ],
  "addresses": [
    "string"
  ],
  "addressType": "IPV4",
  "metadataRequirementList": [
    {
      "unit": "celsius"
    }
  ]
}
```

Listing 3: Payload of an device-query request.

2.2 PUBLISH arrowhead/serviceregistry/management/device-create

Operation: device-create
Input: DeviceListRequest
Output: DeviceListResponse

Creates devices.

```
{
  "devices": [
    {
      "name": "Raspberry1",
      "metadata": {
        "location": "hall-1"
      },
      "addresses": [
        "string"
      ]
    }
  ]
}
```

Listing 4: Payload of an device-create request.

2.3 PUBLISH arrowhead/serviceregistry/management/device-update

Operation: device-update
Input: DeviceListRequest
Output: DeviceListResponse

Updates the metadata and addresses of devices.

```
{
  "devices": [
    {
      "name": "Raspberry1",
      "metadata": {
        "location": "hall-1"
      },
      "addresses": [
        "string"
      ]
    }
  ]
}
```

Listing 5: Payload of an device-update request.

2.4 PUBLISH arrowhead/serviceregistry/management/device-remove

Operation: device-remove
Input: Array<String>

Removes devices by name.

```
[
  "TemperatureSensor"
]
```

Listing 6: Payload of an device-remove request.

2.5 PUBLISH arrowhead/serviceregistry/management/system-query

Operation: system-query
Input: SystemQueryRequest
Output: SystemListResponse

Returns a page of systems matching the filter.
Optional entries of `params`: `verbose`.

```
{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "systemNames": [
    "TemperatureSensor"
  ],
  "addresses": [
    "string"
  ],
  "addressType": "IPV4",
  "metadataRequirementList": [
```

```
    {
      "unit": "celsius"
    }
  ],
  "versions": [
    "1.0.0"
  ],
  "deviceNames": [
    "Raspberry1"
  ]
}
```

Listing 7: Payload of an system-query request.

2.6 PUBLISH arrowhead/serviceregistry/management/system-create

Operation: system-create
Input: SystemListRequest
Output: SystemListResponse

Creates systems on behalf of others.

```
{
  "systems": [
    {
      "name": "TemperatureSensor",
      "metadata": {
        "location": "hall-1"
      },
      "version": "1.0.0",
      "addresses": [
        "string"
      ],
      "deviceName": "Raspberry1"
    }
  ]
}
```

Listing 8: Payload of an system-create request.

2.7 PUBLISH arrowhead/serviceregistry/management/system-update

Operation: system-update
Input: SystemListRequest
Output: SystemListResponse

Updates systems.

```
{
  "systems": [
    {
      "name": "TemperatureSensor",
      "metadata": {
        "location": "hall-1"
      },
      "version": "1.0.0",
      "addresses": [
        "string"
      ],
      "deviceName": "Raspberry1"
    }
  ]
}
```

Listing 9: Payload of an system-update request.

2.8 PUBLISH arrowhead/serviceregistry/management/system-remove

Operation: system-remove
Input: Array<String>

Removes systems by name, with their service instances.

```
[
  "TemperatureSensor"
]
```

Listing 10: Payload of an system-remove request.

2.9 PUBLISH arrowhead/serviceregistry/management/service-definition-query

Operation: service-definition-query
Input: Page
Output: ServiceDefinitionListResponse

Returns a page of service definitions.

```
{
  "page": 0,
  "size": 10,
  "direction": "ASC",
  "sortField": "createdAt"
}
```

Listing 11: Payload of an service-definition-query request.

2.10 PUBLISH arrowhead/serviceregistry/management/service-definition-create

Operation: service-definition-create
Input: ServiceDefinitionListRequest
Output: ServiceDefinitionListResponse

Creates service definitions.

```
{
  "serviceDefinitionNames": [
    "temperatureReading"
  ]
}
```

Listing 12: Payload of an service-definition-create request.

2.11 PUBLISH `arrowhead/serviceregistry/management/service-definition-remove`

Operation: service-definition-remove
Input: Array<String>

Removes service definitions by name, with their service instances.

```
[
  "TemperatureSensor"
]
```

Listing 13: Payload of an service-definition-remove request.

2.12 PUBLISH `arrowhead/serviceregistry/management/service-query`

Operation: service-query
Input: ServiceInstanceQueryRequest
Output: ServiceInstanceListResponse

Returns a page of service instances matching the filter, regardless of the discovery policy.
Optional entries of `params`: `verbose`.

```
{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "instanceIds": [
    "TemperatureSensor|temperatureReading|1.0.0"
  ],
  "providerNames": [
    "TemperatureSensor"
  ],
  "serviceDefinitionNames": [
    "temperatureReading"
  ],
  "versions": [
    "1.0.0"
  ],
  "alivesAt": "2026-10-22T12:00:00Z",
  "metadataRequirementsList": [
    {
      "unit": "celsius"
    }
  ],
  "addressTypes": [
    "IPV4"
  ],
  "interfaceTemplateNames": [
    "string"
  ],
  "interfacePropertyRequirementsList": [
    {
      "unit": "celsius"
    }
  ],
  "policies": [
    "string"
  ]
}
```

Listing 14: Payload of an service-query request.

2.13 PUBLISH arrowhead/serviceregistry/management/service-create

Operation: service-create
Input: ServiceInstanceCreateListRequest
Output: ServiceInstanceListResponse

Creates service instances on behalf of providers. The reservation of core and support service definitions applies.

```
{
  "instances": [
    {
      "systemName": "TemperatureSensor",
      "serviceDefinitionName": "temperatureReading",
      "version": "1.0.0",
      "expiresAt": "2026-10-22T12:00:00Z",
      "metadata": {
        "unit": "celsius"
      },
      "interfaces": [
        {
          "templateName": "generic_https",
          "protocol": "https",
          "policy": "CERT_AUTH",
          "properties": {
            "accessAddresses": [
              "192.168.0.10"
            ],
            "accessPort": 8080,
            "basePath": "/temperature",
            "operations": {
              "read": {
                "method": "GET",
                "path": "/read"
              }
            }
          }
        }
      ]
    }
  ]
}
```

Listing 15: Payload of an service-create request.

2.14 PUBLISH arrowhead/serviceregistry/management/service-update

Operation: service-update
Input: ServiceInstanceUpdateListRequest
Output: ServiceInstanceListResponse

Updates the expiry, metadata and interfaces of service instances.

```
{
  "instances": [
    {
      "instanceId": "TemperatureSensor|temperatureReading|1.0.0",
      "expiresAt": "2026-10-22T12:00:00Z",
      "metadata": {
        "unit": "celsius"
      },
      "interfaces": [
        {
          "templateName": "generic_https",
          "protocol": "https",
```

```
          "policy": "CERT_AUTH",
          "properties": {
            "accessAddresses": [
              "192.168.0.10"
            ],
            "accessPort": 8080,
            "basePath": "/temperature",
            "operations": {
              "read": {
                "method": "GET",
                "path": "/read"
              }
            }
          }
        }
      ]
    }
  ]
}
```

Listing 16: Payload of an service-update request.

2.15 PUBLISH arrowhead/serviceregistry/management/service-remove

Operation: service-remove
Input: Array<String>

Removes service instances by identifier.

```
[
  "string"
]
```

Listing 17: Payload of an service-remove request.

2.16 PUBLISH arrowhead/serviceregistry/management/interface-template-query

Operation: interface-template-query
Input: ServiceInterfaceTemplateQueryRequest
Output: ServiceInterfaceTemplateListResponse

Returns a page of interface templates.

```
{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "templateNames": [
    "generic_https"
  ],
  "protocols": [
    "https"
  ]
}
```

Listing 18: Payload of an interface-template-query request.

2.17 PUBLISH arrowhead/serviceregistry/management/interface-template-create

Operation: interface-template-create
Input: ServiceInterfaceTemplateListRequest
Output: ServiceInterfaceTemplateListResponse

Creates interface templates with their property rules.

```
{
  "interfaceTemplates": [
    {
      "name": "generic_https",
      "protocol": "https",
      "propertyRequirements": [
        {
          "name": "accessPort",
          "mandatory": true,
          "validator": "PORT",
          "validatorParams": [
            "string"
          ]
        }
      ]
    }
  ]
}
```

Listing 19: Payload of an interface-template-create request.

2.18 PUBLISH arrowhead/serviceregistry/management/interface-template-remove

Operation: interface-template-remove
Input: Array<String>

Removes interface templates by name.

```
[
  "TemperatureSensor"
]
```

Listing 20: Payload of an interface-template-remove request.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Each *struct* denotes a JSON Object with the named fields; fields that are not needed may be omitted.

3.1 struct MqttRequestTemplate

The request envelope.

Object Field	Value Type	Description
"traceId"	String	Identifier chosen by the requester, returned in the response.
"authentication"	String	Identity of the requester (see Section 2).
"responseTopic"	String	Topic on which the response is published.
"qosRequirement"	Number	MQTT QoS of the response (0, 1 or 2).
"params"	Object<String>	Optional parameters (the query parameters of the HTTP binding).
"payload"	Object	The operation input.

3.2 struct MqttResponseTemplate

The response envelope.

Object Field	Value Type	Description
"status"	Number	Status code.
"traceId"	String	The trace identifier of the request.
"receiver"	String	Name of the requester.
"payload"	Object	The operation output, or an ErrorMessage.

3.3 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.4 struct Address

A network address with its type.

Object Field	Value Type	Description
"type"	String	Address type: `HOSTNAME`, `IPV4`, `IPV6` or `MAC`.
"address"	String	One network address (host name, IPv4, IPv6 or MAC address).

3.5 struct DeviceListRequest

Devices to create or update.

Object Field	Value Type	Description
"devices"	Array<DeviceRequest>	Devices to create or update.

3.6 struct DeviceListResponse

List of devices.

Object Field	Value Type	Description
"entries"	Array<DeviceResponse>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.7 struct DeviceQueryRequest

Paged filter for devices.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"deviceNames"	Array<String>	Device names to match.
"addresses"	Array<String>	Network addresses of the entity.
"addressType"	String	Address type filter: `HOSTNAME`, `IPV4`, `IPV6` or `MAC`.
"metadataRequirement List"	Array<MetadataRequirement>	Metadata requirements; each map is one alternative.

3.8 struct DeviceRequest

A device and its addresses.

Object Field	Value Type	Description
"name"	String	Device name, PascalCase.
"metadata"	Object	Free-form metadata (JSON object).
"addresses"	Array<String>	Network addresses of the entity.

3.9 struct DeviceResponse

A registered device.

Object Field	Value Type	Description
"name"	String	Device name.
"metadata"	Object	Free-form metadata (JSON object).
"addresses"	Array<Address>	Network addresses with their types.
"createdAt"	DateTime	UTC time of creation.
"updatedAt"	DateTime	UTC time of the last change.

3.10 struct MetadataRequirement

A JSON object of metadata requirements: each key is a metadata path (dot notation), each value the required value or an operator object such as `{"op": "LESS_THAN", "value": 10}`.

3.11 struct Page

Paging and sorting; all fields optional.

Object Field	Value Type	Description
"page"	Number	Zero-based page number.
"size"	Number	Number of records per page (at most `max.page.size`, default 1000).
"direction"	String	Sort direction, `ASC` or `DESC`.
"sortField"	String	Field to sort by.

3.12 struct ServiceDefinitionListRequest

Service definitions to create.

Object Field	Value Type	Description
"serviceDefinition Names"	Array<String>	Service definition names.

3.13 struct ServiceDefinitionListResponse

List of service definitions.

Object Field	Value Type	Description
"entries"	Array<ServiceDefinitionResponse>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.14 struct ServiceDefinitionResponse

A service definition.

Object Field	Value Type	Description
"name"	String	Service definition name.
"createdAt"	DateTime	UTC time of creation.
"updatedAt"	DateTime	UTC time of the last change.

3.15 struct ServiceInstanceCreateListRequest

Service instances to create on behalf of providers.

Object Field	Value Type	Description
"instances"	Array<ServiceInstanceRequest>	Service instances to create or update.

3.16 struct ServiceInstanceInterfaceRequest

One interface of a service instance.

Object Field	Value Type	Description
"templateName"	String	Name of the interface template, snake_case (e.g. `generic_https`).
"protocol"	String	Protocol of the interface, e.g. `http`, `https`, `mqtt`, `mqtts`.
"policy"	String	Security policy of the interface (`NONE`, `CERT_AUTH`, token policies).
"properties"	Object	Interface properties defined by the template, e.g. `accessAddresses`, `accessPort`, `basePath`, `operations`.

3.17 struct ServiceInstanceInterfaceResponse

One interface of a service instance.

Object Field	Value Type	Description
"templateName"	String	Name of the interface template, snake_case (e.g. `generic_https`).
"protocol"	String	Protocol of the interface, e.g. `http`, `https`, `mqtt`, `mqtts`.
"policy"	String	Security policy of the interface: `NONE`, `CERT_AUTH` or one of the token policies.
"properties"	Object	Interface properties defined by the template, e.g. `accessAddresses`, `accessPort`, `basePath`, `operations`.

3.18 struct ServiceInstanceListResponse

List of service instances.

Object Field	Value Type	Description
"entries"	Array<ServiceInstanceResponse>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.19 struct ServiceInstanceQueryRequest

Paged filter for service instances.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"instanceIds"	Array<String>	Instance identifiers to match.
"providerNames"	Array<String>	Provider system names to match.
"serviceDefinition Names"	Array<String>	Service definition names.
"versions"	Array<Version>	Service versions to match.
"alivesAt"	DateTime	Only instances that are still registered (not expired) at this UTC time are matched.
"metadataRequirements List"	Array<MetadataRequirement>	Requirements on the service instance metadata; each map is one alternative.
"addressTypes"	Array<String>	Address types the interfaces must offer (`HOSTNAME`, `IPV4`, `IPV6`, `MAC`).
"interfaceTemplate Names"	Array<String>	Accepted interface templates, e.g. `generic_https`.
"interfaceProperty RequirementsList"	Array<MetadataRequirement>	Requirements on the interface properties; each map is one alternative.
"policies"	Array<String>	Interface security policies to match, e.g. `CERT_AUTH`.

3.20 struct ServiceInstanceRequest

A service instance and its provider.

Object Field	Value Type	Description
"systemName"	String	Name of the provider system.
"serviceDefinitionName"	String	Service definition name, camelCase (e.g. `temperatureReading`).
"version"	Version	Version of the service instance (default `1.0.0`).
"expiresAt"	DateTime	UTC expiry time; no value means no expiry.
"metadata"	Object	Free-form metadata (JSON object).
"interfaces"	Array<ServiceInstanceInterface Request>	Service interfaces offered by the instance.

3.21 struct ServiceInstanceResponse

A registered service instance.

Object Field	Value Type	Description
"instanceId"	String	Service instance identifier `<System>|<serviceDefinition>|<version>`, or policy instance identifier.
"provider"	SystemResponse	The provider system.
"serviceDefinition"	ServiceDefinitionResponse	The service definition.
"version"	Version	Version of the service instance.
"expiresAt"	DateTime	UTC expiry time; no value means no expiry.
"metadata"	Object	Free-form metadata (JSON object).
"interfaces"	Array<ServiceInstanceInterface Response>	Service interfaces offered by the instance.
"createdAt"	DateTime	UTC time of creation.
"updatedAt"	DateTime	UTC time of the last change.

3.22 struct ServiceInstanceUpdateListRequest

Service instances to update.

Object Field	Value Type	Description
"instances"	Array<ServiceInstanceUpdate Request>	Service instances to create or update.

3.23 struct ServiceInstanceUpdateRequest

Changes to one service instance.

Object Field	Value Type	Description
"instanceId"	String	Instance to update.
"expiresAt"	DateTime	UTC expiry time; no value means no expiry.
"metadata"	Object	Free-form metadata (JSON object).
"interfaces"	Array<ServiceInstanceInterface Request>	Service interfaces offered by the instance.

3.24 struct ServiceInterfaceTemplateListRequest

Interface templates to create.

Object Field	Value Type	Description
"interfaceTemplates"	Array<ServiceInterfaceTemplate Request>	Interface templates to create.

3.25 struct ServiceInterfaceTemplateListResponse

List of interface templates.

Object Field	Value Type	Description
"entries"	Array<ServiceInterfaceTemplate Response>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.26 struct ServiceInterfaceTemplateProperty

Rule for one interface property.

Object Field	Value Type	Description
"name"	String	Name of the interface property.
"mandatory"	Boolean	Whether the property must be present in every interface of this template.
"validator"	String	Validator of the value, e.g. `PORT`, `NOT_EMPTY_ADDRESS_LIST`, `HTTP_OPERATIONS`.
"validatorParams"	Array<String>	Parameters of the validator.

3.27 struct ServiceInterfaceTemplateQueryRequest

Paged filter for interface templates.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"templateNames"	Array<String>	Interface template names to match.
"protocols"	Array<String>	Protocols to match.

3.28 struct ServiceInterfaceTemplateRequest

An interface template.

Object Field	Value Type	Description
"name"	String	Template name, snake_case.
"protocol"	String	Protocol of interfaces of this template.
"propertyRequirements"	Array<ServiceInterfaceTemplateProperty>	Property rules of the template.

3.29 struct ServiceInterfaceTemplateResponse

A stored interface template.

Object Field	Value Type	Description
"name"	String	Template name.
"protocol"	String	Protocol of the interface, e.g. `http`, `https`, `mqtt`, `mqtts`.
"propertyRequirements"	Array<ServiceInterfaceTemplateProperty>	Property rules of the template.
"createdAt"	DateTime	UTC time of creation.
"updatedAt"	DateTime	UTC time of the last change.

3.30 struct SystemListRequest

Systems to create or update.

Object Field	Value Type	Description
"systems"	Array<SystemRequest>	Systems to create or update.

3.31 struct SystemListResponse

List of systems.

Object Field	Value Type	Description
"entries"	Array<SystemResponse>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.32 struct SystemQueryRequest

Paged filter for systems.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"systemNames"	Array<String>	System names to match.
"addresses"	Array<String>	Network addresses of the entity.
"addressType"	String	Address type filter: `HOSTNAME`, `IPV4`, `IPV6` or `MAC`.
"metadataRequirementList"	Array<MetadataRequirement>	Metadata requirements; each map is one alternative.
"versions"	Array<Version>	System versions to match.
"deviceNames"	Array<String>	Device names to match.

3.33 struct SystemRequest

A system on behalf of which an operator registers.

Object Field	Value Type	Description
"name"	String	System name, PascalCase.
"metadata"	Object	Free-form metadata (JSON object).
"version"	Version	Version of the system.
"addresses"	Array<String>	Network addresses of the entity.
"deviceName"	String	Name of the device hosting the system.

3.34 struct SystemResponse

A registered system.

Object Field	Value Type	Description
"name"	String	System name.
"metadata"	Object	Free-form metadata (JSON object).
"version"	Version	Version of the system.
"addresses"	Array<Address>	Network addresses with their types.
"device"	DeviceResponse	The device hosting the system, if registered.
"createdAt"	DateTime	UTC time of creation.
"updatedAt"	DateTime	UTC time of the last change.

3.35 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
Boolean	JSON Boolean, `true` or `false`.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.
Version	JSON String, semantic version `major.minor.patch`.

4 References

[1] A. Banks and R. Gupta, “Mqtt version 3.1.1,” OASIS, Standard, 2014.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “serviceRegistryManagement – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.2.2, 2026, serviceRegistry-v5.2.2-serviceRegistryManagement-SD.pdf.

[6] ——, “serviceRegistryManagement HTTP/TLS/JSON – Interface Design Description (ServiceRegistry),” Eclipse Arrowhead, IDD 5.2.2, 2026, serviceRegistry-v5.2.2-serviceRegistryManagement-HTTP-IDD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

sysMonitor AMQP/TLS/JSON

Interface Design Description

Service ID: *"sysMonitor"*

Abstract
This document describes the AMQP 0-9-1 protocol with TLS payload security and JSON payload encoding variant of the `sysMonitor` service of the ServiceRegistry system (SysMonitor-AMQP-IDD-5.2.2).

Contents

1 Overview **3**
1.1 Interface registration 3

2 Service Operations **4**
2.1 operation echo 4
2.2 operation power 4
2.3 operation status 4

3 Data Models **6**
3.1 struct Observation 6
3.2 Primitives 6

4 References **7**

5 Revision History **8**
5.1 Amendments 8
5.2 Quality Assurance 8

1 Overview

This document describes the AMQP binding of the native `sysMonitor` service of ServiceRegistry.
It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	AMQP	0-9-1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `sysMonitor` service interface

This document provides the Interface Design Description IDD to the *sysMonitor – Service Description* document [5].

1.1 Interface registration

With `sysmonitor.enabled=true`, ServiceRegistry registers the service at start-up with interface template `generic_amqps` (protocol `amqps`, policy `CERT_AUTH`; `generic_amqp` without TLS) and the following properties and metadata:

```
{
  "serviceDefinitionName": "sysMonitor",
  "version": "1.0.0",
  "metadata": {
    "contract": "SysMonitor-AMQP-IDD-5.2.2",
    "schemaVersion": "1.0",
    "publisherMode": "native",
    "observationTypes": [
      "echo",
      "power",
      "status"
    ],
    "delivery": "persistent, publisher-confirmed"
  },
  "interfaces": [
    {
      "templateName": "generic_amqps",
      "protocol": "amqps",
      "policy": "CERT_AUTH",
      "properties": {
        "accessAddresses": [
          "amqp-broker"
        ],
        "accessPort": 5671,
        "exchange": "arrowhead",
        "baseRoutingKey": "TestCloud.sysmonitor.ServiceRegistry.ServiceRegistry",
        "operations": [
          "echo",
          "power",
          "status"
        ]
      }
    }
  ]
}
```

Listing 1: Registered service instance (TestCloud).

2 Service Operations

The publisher connects to the broker (`sysmonitor.broker.address`, default `amqp-broker`; port `sysmonitor.broker.port`, default 5671) with TLS, verifies the broker host name and authenticates with its X.509 system certificate (SASL `EXTERNAL`; the certificate CN is the broker user name). Every publication interval it publishes each observation to the topic exchange `arrowhead` with the routing key `<baseRoutingKey>.<operation>`, where the base routing key is `<Cloud>.sysmonitor.<System>.<serviceInstance>`. The local cloud name is taken from `sysmonitor.local.cloud` or the system certificate. Messages are persistent (delivery mode 2), `application/json`, carry the observation identifier as message id, and are publisher-confirmed (timeout 5 s); a failed cycle is retried with a new connection at the next interval. The broker's topic permissions allow the system to publish only below its own base routing key.

2.1 PUBLISH `<Cloud>.sysmonitor.ServiceRegistry.ServiceRegistry.echo`

Operation: echo
Output: Observation

Liveness: `status` (`alive` or `degraded`), `uptimeSeconds` and a `detail` text; quality `good` or `uncertain`.

2.2 PUBLISH `<Cloud>.sysmonitor.ServiceRegistry.ServiceRegistry.power`

Operation: power
Output: Observation

Power supply: `supplyType=line`, `state=available`; unit `1`.

2.3 PUBLISH `<Cloud>.sysmonitor.ServiceRegistry.ServiceRegistry.status`

Operation: status
Output: Observation

Process state: `state=running`, `health` (`healthy`/`degraded`) and `serviceCount`, the number of registered services.

```
{
  "schemaVersion": "1.0",
  "observationId": "ServiceRegistry-echo-1792670400000-42",
  "providerSystem": "ServiceRegistry",
  "serviceInstance": "ServiceRegistry",
  "observationType": "echo",
  "sourceTime": "2026-10-22T12:00:00.000Z",
  "sequence": 42,
  "value": {
    "status": "alive",
    "uptimeSeconds": 3600,
    "detail": "ServiceRegistry is running"
  },
  "quality": "good",
  "metadata": {
    "publisherMode": "native"
  }
}
```

Listing 2: An echo observation.

3 Data Models

3.1 struct Observation

Object Field	Value Type	Description
"schemaVersion"	String	`1.0`.
"observationId"	String	`<System>-<type>-<epoch ms>-<sequence>`.
"providerSystem"	String	The publishing system.
"serviceInstance"	String	Service instance, default the system name.
"observationType"	String	`echo`, `power` or `status`.
"sourceTime"	String	ISO 8601 UTC time with milliseconds.
"sequence"	Number	Sequence number per publisher.
"value"	Object	Observed value, see the operations.
"unit"	String	`1` for `power` and `status`; absent for `echo`.
"quality"	String	`good` or `uncertain`.
"metadata"	Object	`{"publisherMode": "native"}`.

3.2 Primitives

Type	Description
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] AMQP Working Group, “Amqp 0-9-1 specification,” AMQP Working Group, Tech. Rep., 2008.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “sysMonitor – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.2.2, 2026, serviceRegistry-v5.2.2-sysMonitor-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

systemDiscovery HTTP/TLS/JSON

Interface Design Description

Service ID: *"systemDiscovery"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `systemDiscovery` service of the ServiceRegistry system.

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation register . . . 4
2.2 operation lookup . . . 5
2.3 operation revoke . . . 6

3 Data Models **7**
3.1 struct Address . . . 7
3.2 struct DeviceResponse . . . 7
3.3 struct ErrorMessage . . . 7
3.4 struct MetadataRequirement . . . 7
3.5 struct SystemListResponse . . . 7
3.6 struct SystemLookupRequest . . . 7
3.7 struct SystemRegisterRequest . . . 8
3.8 struct SystemResponse . . . 8
3.9 Primitives . . . 8

4 References **10**

5 Revision History **11**
5.1 Amendments . . . 11
5.2 Quality Assurance . . . 11

1 Overview

This document describes the `systemDiscovery` service interface of the ServiceRegistry system, which provides registration and lookup of the application, support and core systems of the local cloud, with their addresses, version, metadata and hosting device.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `systemDiscovery` service interface

This document provides the Interface Design Description IDD to the *systemDiscovery – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `systemDiscovery` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https` (`generic_http` when TLS is disabled), access port 8443 and base path `/serviceregistry/system-discovery`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`register`	POST	`/register`
`lookup`	POST	`/lookup`
`revoke`	DELETE	`/revoke`

2 Service Operations

The interfaces of the `systemDiscovery` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `systemDiscovery` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json`. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

2.1 POST /serviceregistry/system-discovery/register

Operation: register
Input: SystemRegisterRequest
Output: SystemResponse

Registers the requesting system (name from its identity) with addresses, version, metadata and an optional device. Returns 201 for a new entry, 200 if an identical entry exists; an existing system with different data is refused (400).

A successful call returns `201 Created` for a new entry, `200 OK` if an identical entry already existed with the output in the body.

```
POST /serviceregistry/system-discovery/register HTTP/1.1
Host: serviceregistry:8443
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "metadata": {
    "location": "hall-1"
  },
  "version": "1.0.0",
  "addresses": [
    "string"
  ],
  "deviceName": "Raspberry1"
}
```

Listing 1: An register invocation.

```
{
  "name": "TemperatureSensor",
  "metadata": {
    "location": "hall-1"
  },
  "version": "1.0.0",
  "addresses": [
    {
      "type": "IPV4",
      "address": "192.168.0.10"
    }
  ],
  "device": {
    "name": "Raspberry1",
    "metadata": {
```

```
      "location": "hall-1"
    },
    "addresses": [
      {
        "type": "IPV4",
        "address": "192.168.0.10"
      }
    ],
    "createdAt": "2026-10-22T12:00:00Z",
    "updatedAt": "2026-10-22T12:00:00Z"
  },
  "createdAt": "2026-10-22T12:00:00Z",
  "updatedAt": "2026-10-22T12:00:00Z"
}
```

Listing 2: An register response.

2.2 POST /serviceregistry/system-discovery/lookup?verbose=*{verbose}*

Operation: lookup
Input: SystemLookupRequest
Output: SystemListResponse

Returns the systems matching all given filters. With `verbose=true` (and `discovery.verbose` enabled) the device details are included.

Query parameters: *verbose* (boolean, optional, default `false`). List values are repeated or comma-separated.

A successful call returns `200 OK` with the output in the body.

```
POST /serviceregistry/system-discovery/lookup?verbose=<verbose> HTTP/1.1
Host: serviceregistry:8443
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "systemNames": [
    "TemperatureSensor"
  ],
  "addresses": [
    "string"
  ],
  "addressType": "IPV4",
  "metadataRequirementList": [
    {
      "unit": "celsius"
    }
  ],
  "versions": [
    "1.0.0"
  ],
  "deviceNames": [
    "Raspberry1"
  ]
}
```

Listing 3: An lookup invocation.

```
{
  "entries": [
    {
      "name": "TemperatureSensor",
      "metadata": {
        "location": "hall-1"
```

```
      },
      "version": "1.0.0",
      "addresses": [
        {
          "type": "IPV4",
          "address": "192.168.0.10"
        }
      ],
      "device": {
        "name": "Raspberry1",
        "metadata": {
          "location": "hall-1"
        },
        "addresses": [
          {}
        ],
        "createdAt": "2026-10-22T12:00:00Z",
        "updatedAt": "2026-10-22T12:00:00Z"
      },
      "createdAt": "2026-10-22T12:00:00Z",
      "updatedAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```


Listing 4: An lookup response.

2.3 DELETE /serviceregistry/system-discovery/revoke

Operation: revoke

Removes the requesting system and its service instances. Returns 200 if removed, 204 if it was not registered.

A successful call returns `200 OK` if the entry was removed, `204 No Content` if there was nothing to remove.

```
DELETE /serviceregistry/system-discovery/revoke HTTP/1.1
Host: serviceregistry:8443
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 5: An revoke invocation.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.9, which are used to represent things like hashes, identifiers and texts.

3.1 struct Address

A network address with its type.

Object Field	Value Type	Description
"type"	String	Address type: `HOSTNAME`, `IPV4`, `IPV6` or `MAC`.
"address"	String	One network address (host name, IPv4, IPv6 or MAC address).

3.2 struct DeviceResponse

A registered device.

Object Field	Value Type	Description
"name"	String	Device name.
"metadata"	Object	Free-form metadata (JSON object).
"addresses"	Array<Address>	Network addresses with their types.
"createdAt"	DateTime	UTC time of creation.
"updatedAt"	DateTime	UTC time of the last change.

3.3 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.4 struct MetadataRequirement

A JSON object of metadata requirements: each key is a metadata path (dot notation), each value the required value or an operator object such as `{"op": "LESS_THAN", "value": 10}`.

3.5 struct SystemListResponse

List of systems.

Object Field	Value Type	Description
"entries"	Array<SystemResponse>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.6 struct SystemLookupRequest

Filter for systems; all fields optional, combined with AND.

Object Field	Value Type	Description
"systemNames"	Array<String>	System names to match.
"addresses"	Array<String>	Network addresses of the entity.
"addressType"	String	Address type filter: `HOSTNAME`, `IPV4`, `IPV6` or `MAC`.
"metadataRequirementList"	Array<MetadataRequirement>	Metadata requirements; each map is one alternative.
"versions"	Array<Version>	System versions to match.
"deviceNames"	Array<String>	Device names to match.

3.7 struct SystemRegisterRequest

Registers the requesting system; the name is taken from its authenticated identity.

Object Field	Value Type	Description
"metadata"	Object	Free-form metadata (JSON object).
"version"	Version	Version of the system.
"addresses"	Array<String>	Network addresses of the entity.
"deviceName"	String	Name of the device hosting the system.

3.8 struct SystemResponse

A registered system.

Object Field	Value Type	Description
"name"	String	System name.
"metadata"	Object	Free-form metadata (JSON object).
"version"	Version	Version of the system.
"addresses"	Array<Address>	Network addresses with their types.
"device"	DeviceResponse	The device hosting the system, if registered.
"createdAt"	DateTime	UTC time of creation.
"updatedAt"	DateTime	UTC time of the last change.

3.9 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.
Version	JSON String, semantic version `major.minor.patch`.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “systemDiscovery – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.2.2, 2026, serviceRegistry-v5.2.2-systemDiscovery-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

systemDiscovery MQTT/TLS/JSON

Interface Design Description

Service ID: *"systemDiscovery"*

Abstract
This document describes the MQTT protocol with TLS payload security and JSON payload encoding variant of the `systemDiscovery` service of the ServiceRegistry system.

Contents

1 Overview **3**

2 Service Operations **4**
2.1 operation register . . . 4
2.2 operation lookup . . . 4
2.3 operation revoke . . . 5

3 Data Models **6**
3.1 struct MqttRequestTemplate . . . 6
3.2 struct MqttResponseTemplate . . . 6
3.3 struct ErrorMessage . . . 6
3.4 struct Address . . . 6
3.5 struct DeviceResponse . . . 6
3.6 struct MetadataRequirement . . . 7
3.7 struct SystemListResponse . . . 7
3.8 struct SystemLookupRequest . . . 7
3.9 struct SystemRegisterRequest . . . 7
3.10 struct SystemResponse . . . 7
3.11 Primitives . . . 8

4 References **9**

5 Revision History **10**
5.1 Amendments . . . 10
5.2 Quality Assurance . . . 10

1 Overview

This document describes the `systemDiscovery` service interface of the ServiceRegistry system over MQTT, which provides registration and lookup of the application, support and core systems of the local cloud, with their addresses, version, metadata and hosting device.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	MQTT	3.1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `systemDiscovery` service interface

This document provides the Interface Design Description IDD to the *systemDiscovery – Service Description* document [5]. The operations, their semantics and the data models are the same as in the HTTP/TLS/JSON IDD [6]; this document specifies the MQTT binding.

The MQTT interface is offered only when the system runs with `mqtt.api.enabled=true`; it is then registered with the interface template `generic_mqtts` (`generic_mqtt` without TLS), the broker address and port (`mqtt.broker.address`, `mqtt.broker.port`) and the base topic `arrowhead/serviceregistry/system-discovery/`. In the Arrowhead-test-cloud-java deployment the MQTT interfaces of the core systems are disabled.

2 Service Operations

The provider subscribes to one topic per operation, the base topic followed by the operation name:

Operation	Request topic
`register`	`arrowhead/serviceregistry/system-discovery/register`
`lookup`	`arrowhead/serviceregistry/system-discovery/lookup`
`revoke`	`arrowhead/serviceregistry/system-discovery/revoke`

A consumer publishes a request envelope (MqttRequestTemplate) to the topic and receives the answer (MqttResponseTemplate) on its own `responseTopic`. The `authentication` field carries the requester's identity according to the provider's authentication policy: `IDENTITY-TOKEN//<token>` (outsourced or internal policy), `SYSTEM//<name>` (declared policy) or the requester's X.509 certificate (certificate policy). The `status` of the response uses the HTTP status code numbers (200, 201, 204, 400, 401, 403, 404, 408, 423, 500, 503); on error the payload is an ErrorMessage.

```
{
  "traceId": "a1b2c3",
  "authentication": "IDENTITY-TOKEN//<token>",
  "responseTopic": "arrowhead/response/TemperatureDisplay/a1b2c3",
  "qosRequirement": 1,
  "params": {},
  "payload": {}
}
```

Listing 1: Request envelope.

```
{
  "status": 200,
  "traceId": "a1b2c3",
  "receiver": "TemperatureDisplay",
  "payload": {}
}
```

Listing 2: Response envelope.

2.1 PUBLISH `arrowhead/serviceregistry/system-discovery/register`

Operation: register
Input: SystemRegisterRequest
Output: SystemResponse

Registers the requesting system (name from its identity) with addresses, version, metadata and an optional device. Returns 201 for a new entry, 200 if an identical entry exists; an existing system with different data is refused (400).

```
{
  "metadata": {
    "location": "hall-1"
  },
  "version": "1.0.0",
  "addresses": [
    "string"
  ],
  "deviceName": "Raspberry1"
}
```

Listing 3: Payload of an register request.

2.2 PUBLISH `arrowhead/serviceregistry/system-discovery/lookup`

Operation: lookup
Input: SystemLookupRequest
Output: SystemListResponse

Returns the systems matching all given filters. With `verbose=true` (and `discovery.verbose` enabled) the device details are included.

Optional entries of `params`: `verbose`.

```
{
  "systemNames": [
    "TemperatureSensor"
  ],
  "addresses": [
    "string"
  ],
  "addressType": "IPV4",
  "metadataRequirementList": [
    {
      "unit": "celsius"
    }
  ],
  "versions": [
    "1.0.0"
  ],
  "deviceNames": [
    "Raspberry1"
  ]
}
```

Listing 4: Payload of an lookup request.

2.3 PUBLISH `arrowhead/serviceregistry/system-discovery/revoke`

Operation: revoke

Removes the requesting system and its service instances. Returns 200 if removed, 204 if it was not registered.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Each *struct* denotes a JSON Object with the named fields; fields that are not needed may be omitted.

3.1 struct MqttRequestTemplate

The request envelope.

Object Field	Value Type	Description
"traceId"	String	Identifier chosen by the requester, returned in the response.
"authentication"	String	Identity of the requester (see Section 2).
"responseTopic"	String	Topic on which the response is published.
"qosRequirement"	Number	MQTT QoS of the response (0, 1 or 2).
"params"	Object<String>	Optional parameters (the query parameters of the HTTP binding).
"payload"	Object	The operation input.

3.2 struct MqttResponseTemplate

The response envelope.

Object Field	Value Type	Description
"status"	Number	Status code.
"traceId"	String	The trace identifier of the request.
"receiver"	String	Name of the requester.
"payload"	Object	The operation output, or an ErrorMessage.

3.3 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.4 struct Address

A network address with its type.

Object Field	Value Type	Description
"type"	String	Address type: `HOSTNAME`, `IPV4`, `IPV6` or `MAC`.
"address"	String	One network address (host name, IPv4, IPv6 or MAC address).

3.5 struct DeviceResponse

A registered device.

Object Field	Value Type	Description
"name"	String	Device name.
"metadata"	Object	Free-form metadata (JSON object).
"addresses"	Array<Address>	Network addresses with their types.
"createdAt"	DateTime	UTC time of creation.
"updatedAt"	DateTime	UTC time of the last change.

3.6 struct MetadataRequirement

A JSON object of metadata requirements: each key is a metadata path (dot notation), each value the required value or an operator object such as `{"op": "LESS_THAN", "value": 10}`.

3.7 struct SystemListResponse

List of systems.

Object Field	Value Type	Description
"entries"	Array<SystemResponse>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.8 struct SystemLookupRequest

Filter for systems; all fields optional, combined with AND.

Object Field	Value Type	Description
"systemNames"	Array<String>	System names to match.
"addresses"	Array<String>	Network addresses of the entity.
"addressType"	String	Address type filter: `HOSTNAME`, `IPV4`, `IPV6` or `MAC`.
"metadataRequirementList"	Array<MetadataRequirement>	Metadata requirements; each map is one alternative.
"versions"	Array<Version>	System versions to match.
"deviceNames"	Array<String>	Device names to match.

3.9 struct SystemRegisterRequest

Registers the requesting system; the name is taken from its authenticated identity.

Object Field	Value Type	Description
"metadata"	Object	Free-form metadata (JSON object).
"version"	Version	Version of the system.
"addresses"	Array<String>	Network addresses of the entity.
"deviceName"	String	Name of the device hosting the system.

3.10 struct SystemResponse

A registered system.

Object Field	Value Type	Description
"name"	String	System name.
"metadata"	Object	Free-form metadata (JSON object).
"version"	Version	Version of the system.
"addresses"	Array<Address>	Network addresses with their types.
"device"	DeviceResponse	The device hosting the system, if registered.
"createdAt"	DateTime	UTC time of creation.
"updatedAt"	DateTime	UTC time of the last change.

3.11 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.
Version	JSON String, semantic version `major.minor.patch`.

4 References

[1] A. Banks and R. Gupta, “Mqtt version 3.1.1,” OASIS, Standard, 2014.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “systemDiscovery – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.2.2, 2026, serviceRegistry-v5.2.2-systemDiscovery-SD.pdf.

[6] ——, “systemDiscovery HTTP/TLS/JSON – Interface Design Description (ServiceRegistry),” Eclipse Arrowhead, IDD 5.2.2, 2026, serviceRegistry-v5.2.2-systemDiscovery-HTTP-IDD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

1 Interface overview

Mandatory mutual TLS, Arrowhead token/certificate binding, JSON request and response documents. Echo responses, orchestration subscription IDs, authorization public keys and encryption IV responses are text. Certificate-only operations and management are distinguished in the table.

1.1 HTTPS interface operations

HTTPS uses JSON unless an operation specifies text. Request bodies are limited to 1 MiB; upstream responses to 8 MiB. Identity-token format: `Bearer IDENTITY-TOKEN//token`. Certificate-only identity operations still require verified mutual TLS and apply credential/certificate binding.

Method	Path	Access
GET	/serviceregistry/general/mgmt/get-config	management
POST	/serviceregistry/general/mgmt/logs	management
GET	/serviceregistry/monitor/echo	certificate
POST	/serviceregistry/device-discovery/register	identity
POST	/serviceregistry/device-discovery/lookup	identity
DELETE	/serviceregistry/device-discovery/revoke/{name}	identity
POST	/serviceregistry/system-discovery/register	identity
POST	/serviceregistry/system-discovery/lookup	identity
DELETE	/serviceregistry/system-discovery/revoke	identity
POST	/serviceregistry/service-discovery/register	identity
POST	/serviceregistry/service-discovery/lookup	identity
DELETE	/serviceregistry/service-discovery/revoke/{instanceId}	identity
POST	/serviceregistry/mgmt/devices/query	management
POST	/serviceregistry/mgmt/devices	management
DELETE	/serviceregistry/mgmt/devices	management
PUT	/serviceregistry/mgmt/devices	management
POST	/serviceregistry/mgmt/systems/query	management
POST	/serviceregistry/mgmt/systems	management
DELETE	/serviceregistry/mgmt/systems	management
PUT	/serviceregistry/mgmt/systems	management
POST	/serviceregistry/mgmt/service-definitions/query	management
POST	/serviceregistry/mgmt/service-definitions	management
DELETE	/serviceregistry/mgmt/service-definitions	management
POST	/serviceregistry/mgmt/service-instances/query	management
POST	/serviceregistry/mgmt/service-instances	management
DELETE	/serviceregistry/mgmt/service-instances	management
PUT	/serviceregistry/mgmt/service-instances	management
POST	/serviceregistry/mgmt/interface-templates/query	management
POST	/serviceregistry/mgmt/interface-templates	management

Method	Path	Access
DELETE	/serviceregistry/mgmt/interface-templates	management

Errors contain errorMessage, errorCode, exceptionType, origin and timestamp. Exact validation/error parity and all Java security-policy settings remain release gates. Unknown routes return 404; unavailable authorities fail closed.

2 Limitations

Registry records, interface templates, expiry, cascades, authorized discovery and management are implemented. Java SQL import, the full property-validator catalogue, normalization, advanced version/time/address matching and all discovery-policy options need additional work. Unsupported custom validators fail explicitly; they are not silently accepted.